

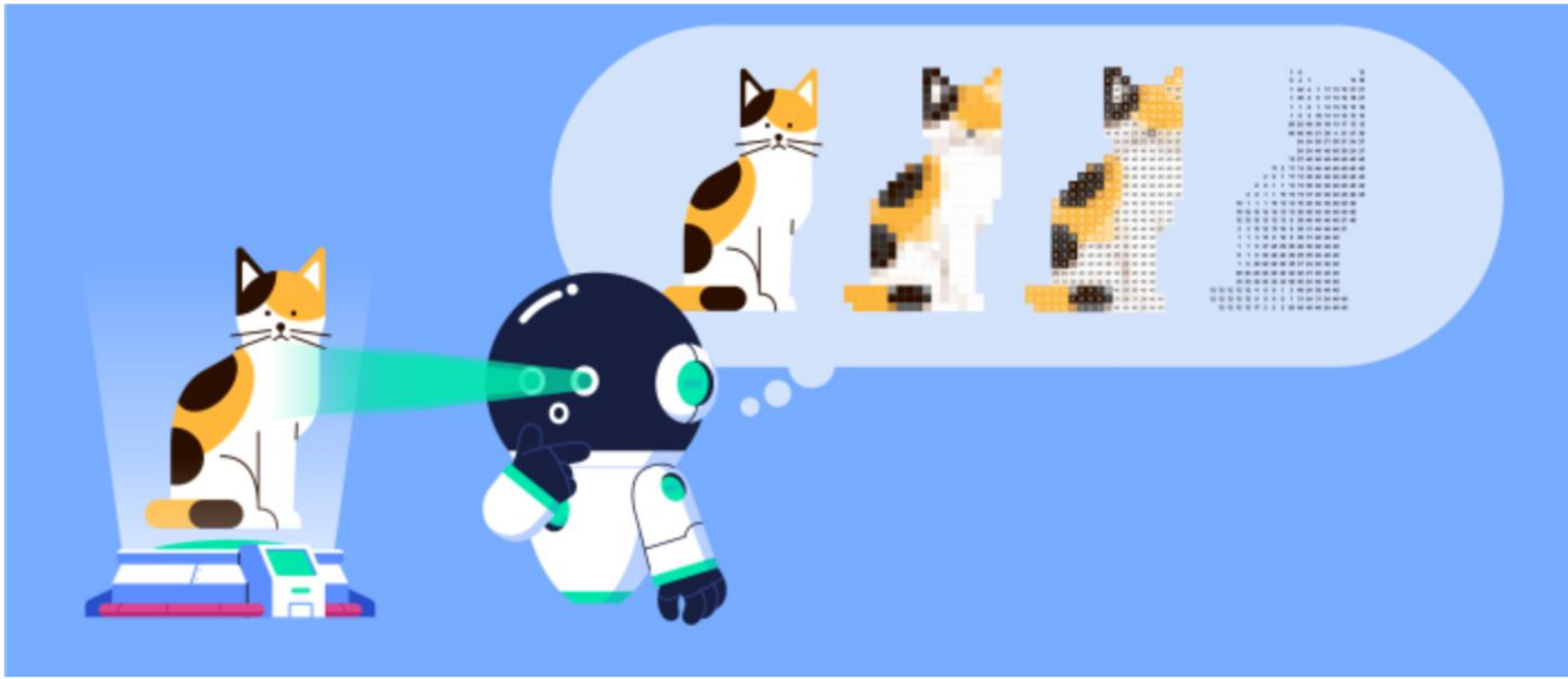
반도체장비예지보전실무 05

이미지 제조 데이터 기반 AI 개발

이미지 데이터의 이해

이미지 → 데이터

- 컴퓨터는 숫자만 알아볼 수 있음
- 이미지 역시 숫자로 표현되어야 컴퓨터가 처리할 수 있음



이미지 → 데이터

- 이미지를 픽셀 형태로 만들어서 처리함
- 이미지는 작은 사각형들로 이루어지는데, 이 사각형 한 개를 '픽셀'이라 함



픽셀 (pixel, 화소)

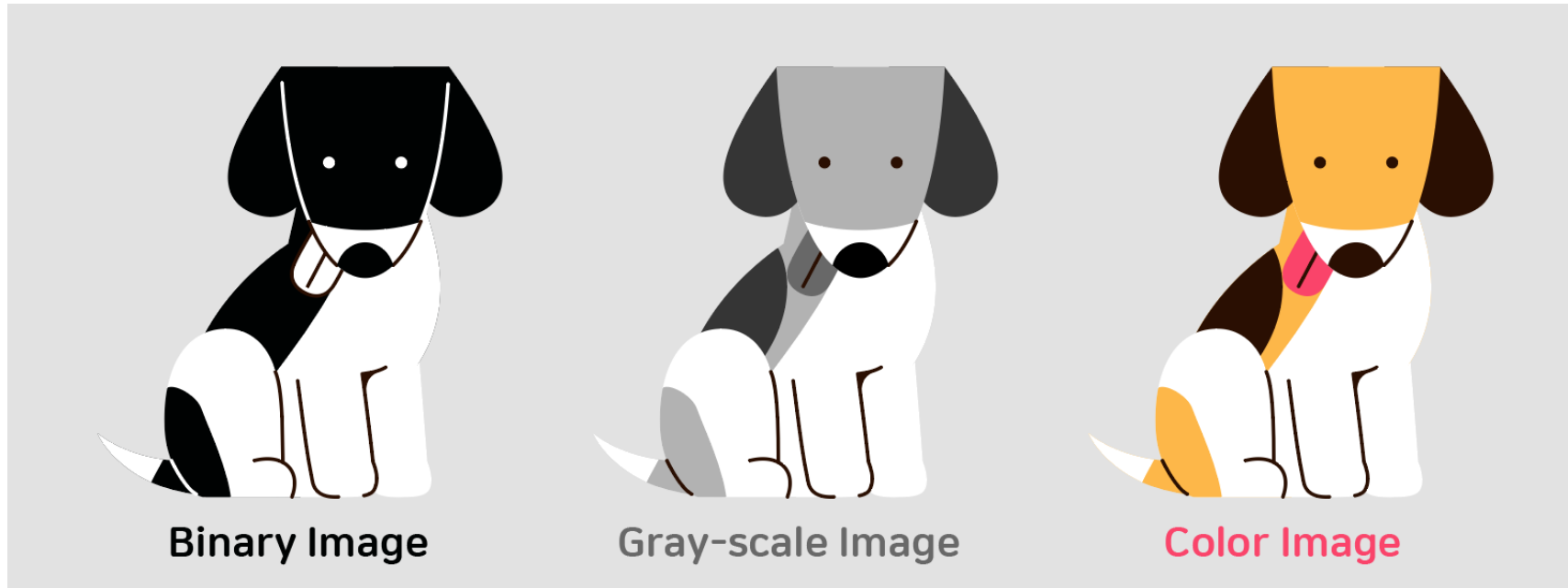
- 디지털 화면을 구성하는 최소 단위로, 사각형의 점.
- Picture Element의 줄임말



픽셀의 수치화

= 이미지가 컴퓨터의 세계에 존재하는 방법

이미지 데이터



Binary Image

Gray-scale Image

Color Image

0 또는 1

0~255

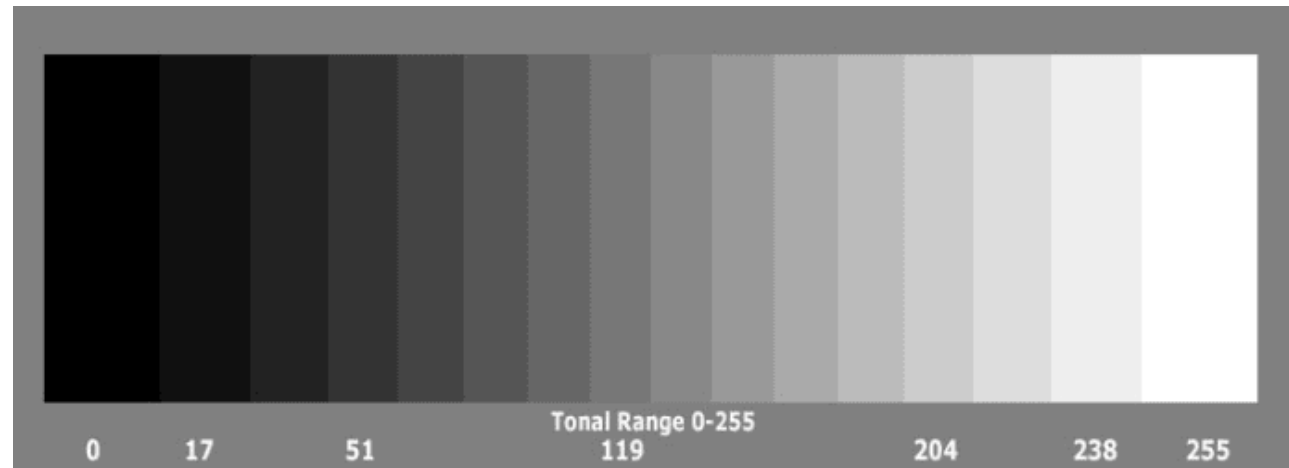
빨강(Red) 0~255

초록(Green) 0~255

파랑(Blue) 0~255

흑백 이미지의 수치화: Grayscale

- 컬러 정보가 없는 무채색 계열의 이미지 (흑백 이미지)
- 픽셀당 일반적으로 8비트를 사용하여 0(검정)부터 255(흰색)까지의 밝기 단계를 표현

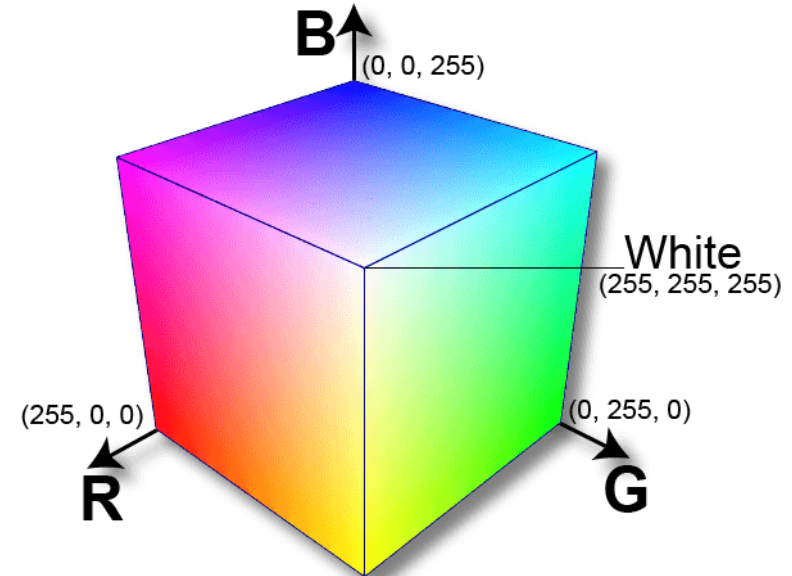
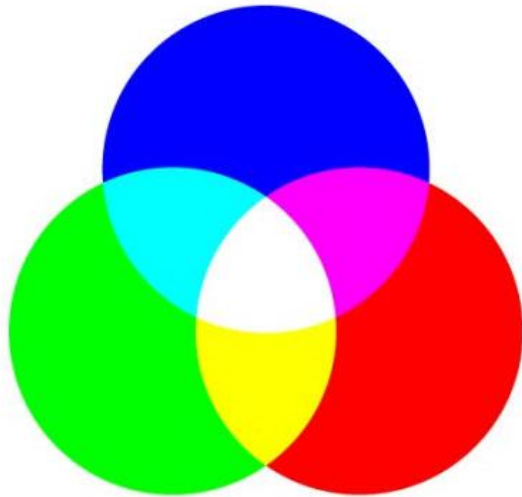


흑백 이미지의 수치화: Grayscale



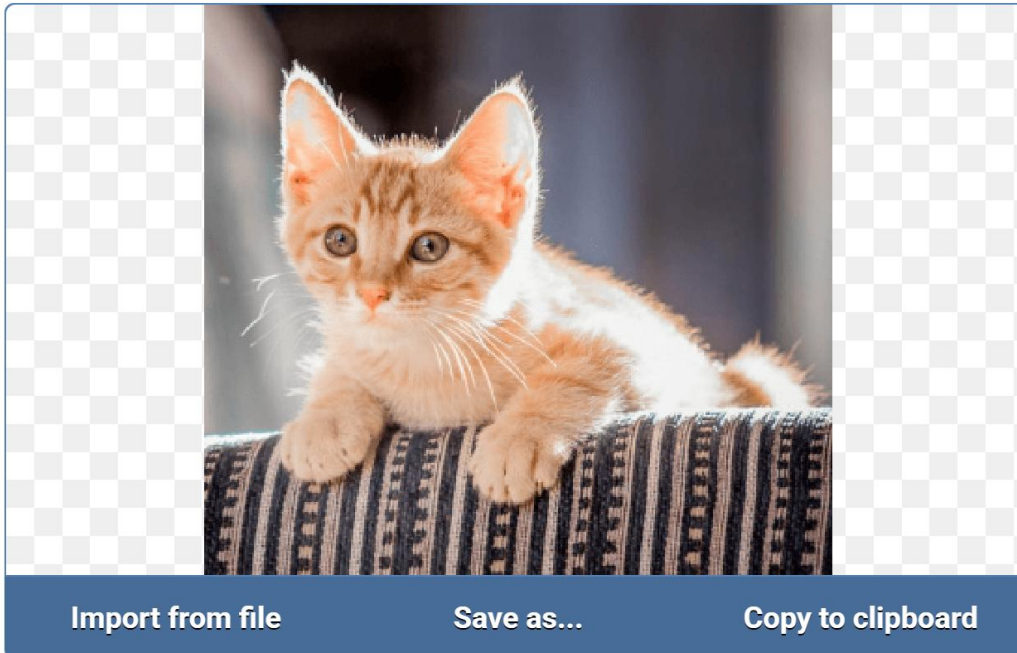
컬러 이미지의 수치화: RGB

- 빛의 3원색인 Red(빨강), Green(초록), Blue의 약자
- 세 가지 빛을 혼합하여 다양한 색상을 표현하는 디지털 색상 모델
- 일반적으로 0~255 사이의 숫자로 각 색상의 강도를 표현

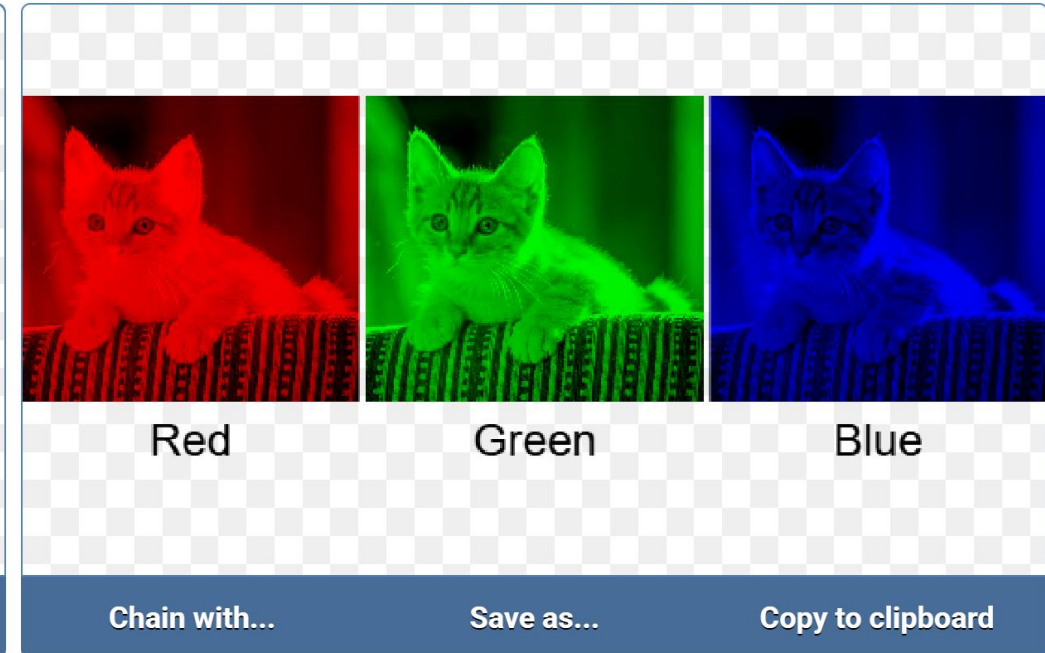


컬러 이미지의 RGB Channels

Input PNG ?



PNG Color Channels



컬러 이미지의 RGB Channels

Input PNG ?

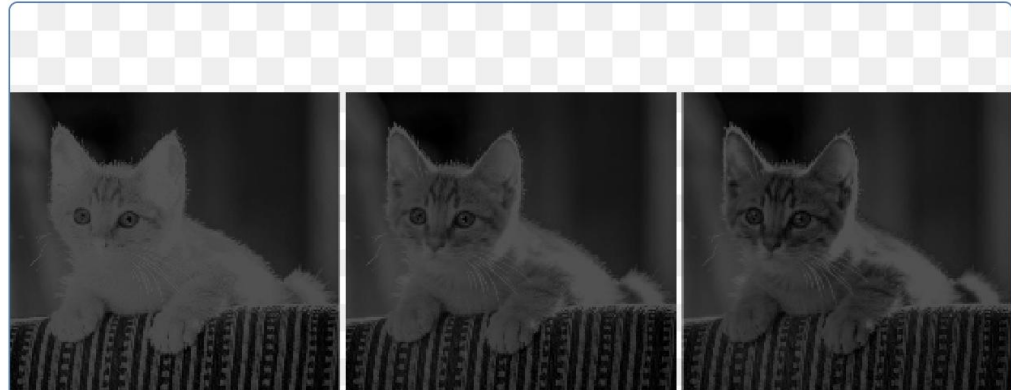


Import from file

Save as...

Copy to clipboard

PNG Color Channels



Red

Green

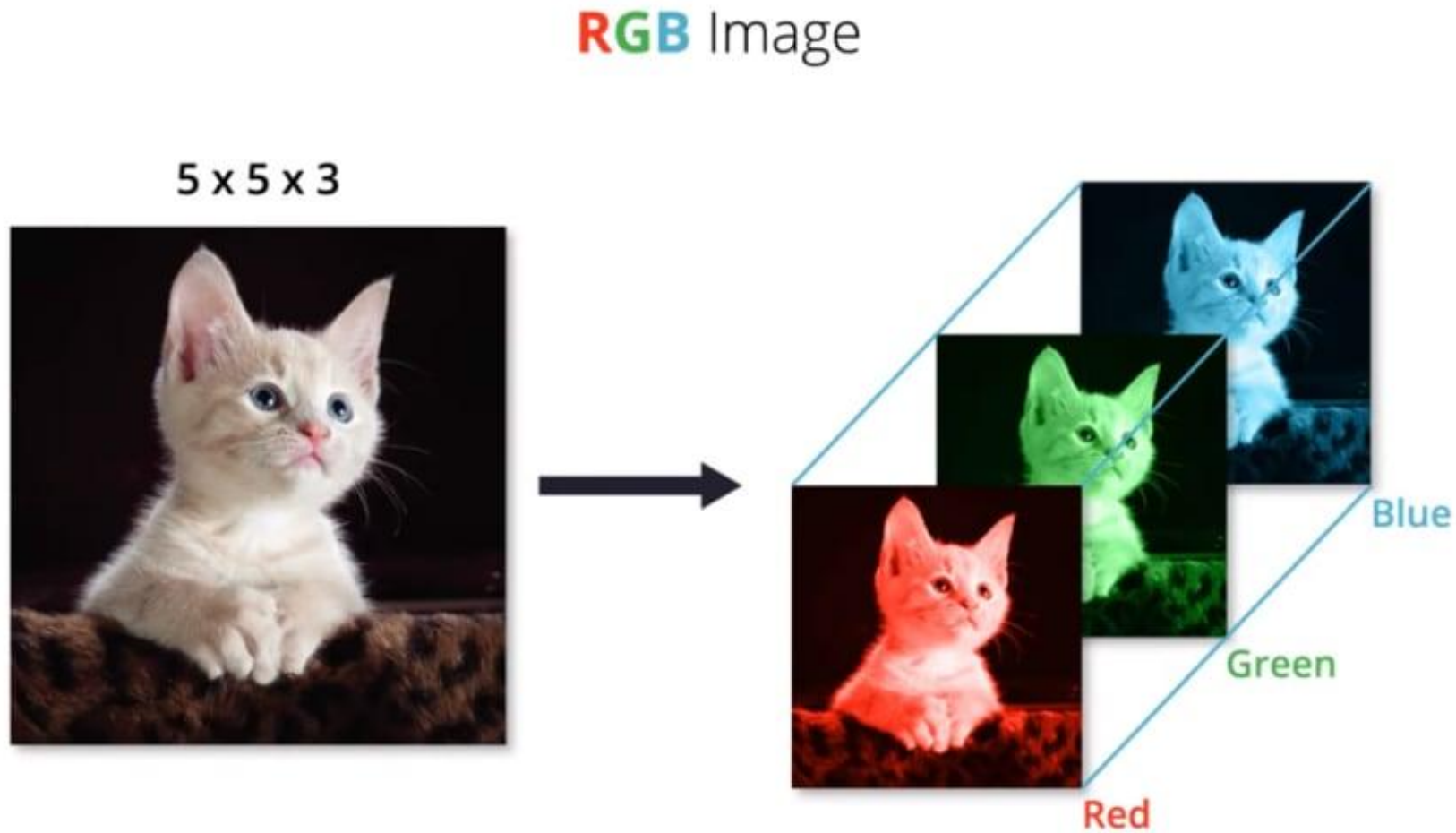
Blue

Chain with...

Save as...

Copy to clipboard

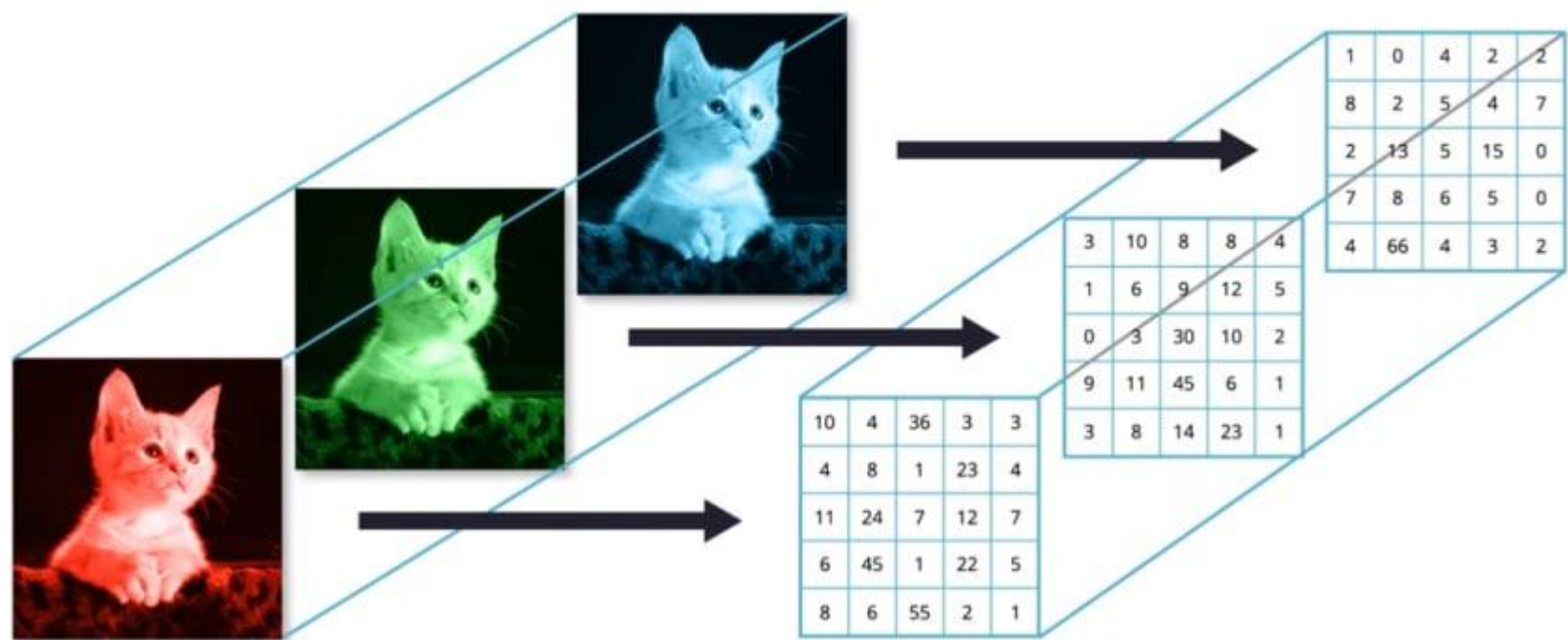
컬러 이미지의 내부적 표현



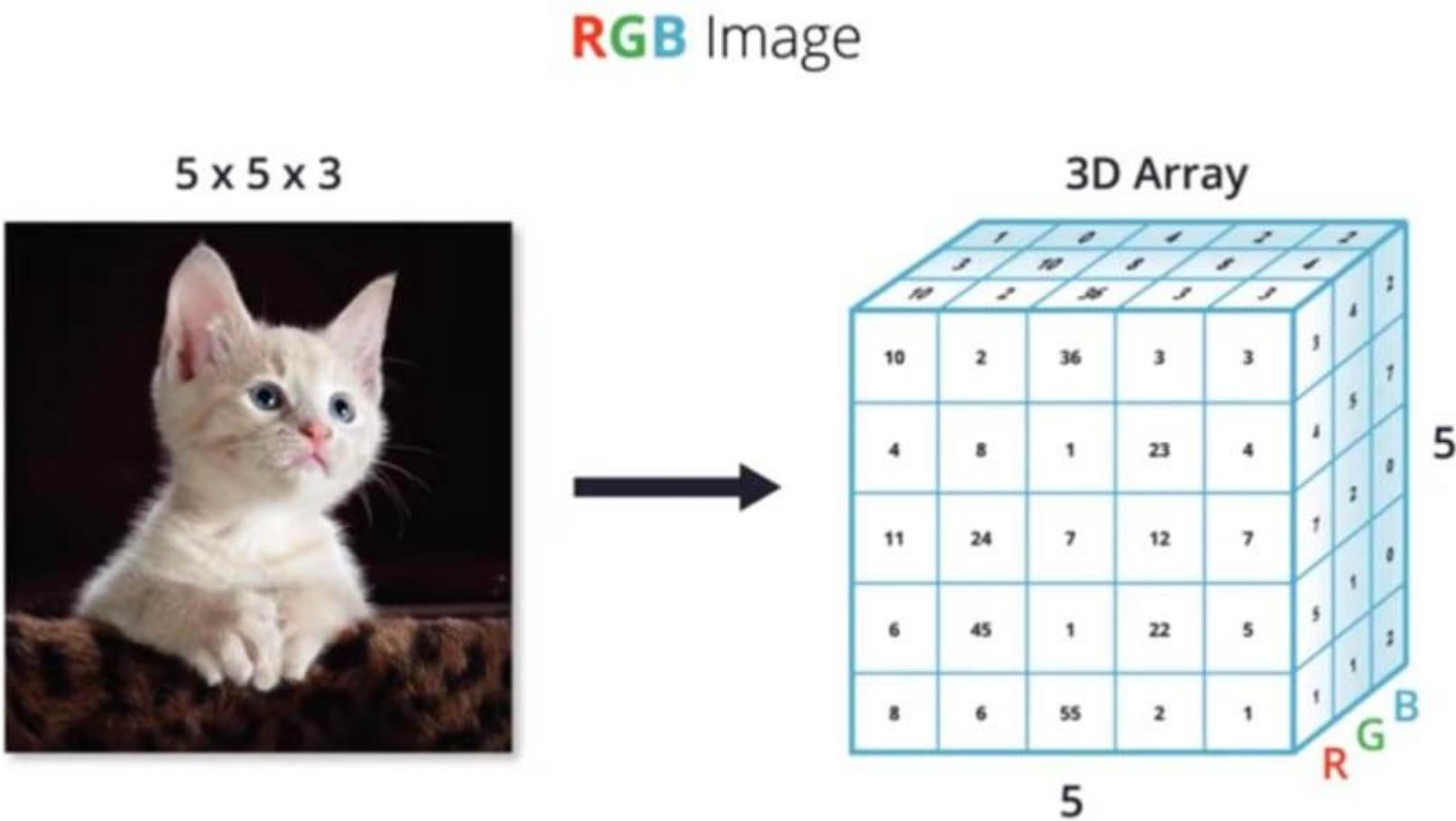
컬러 이미지의 내부적 표현



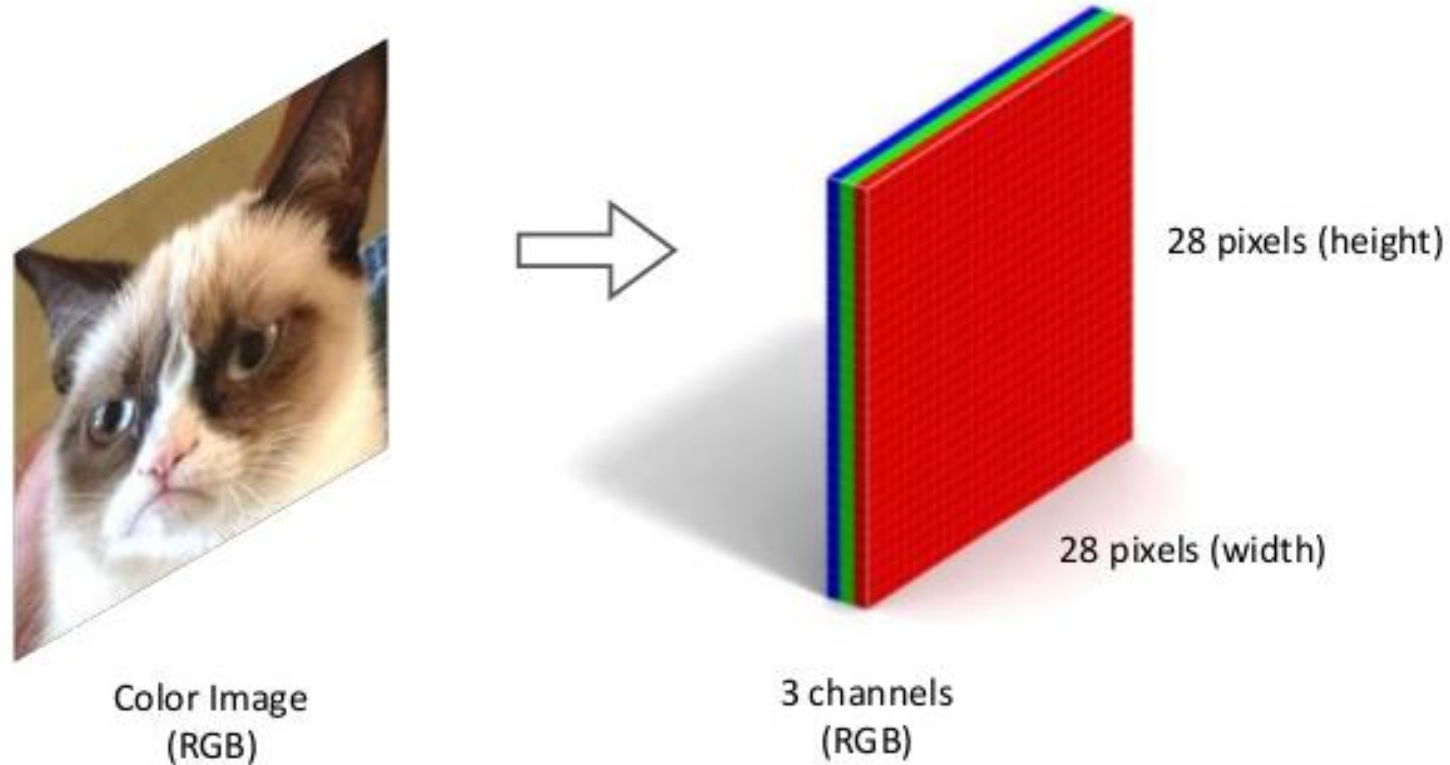
RGB Image



컬러 이미지의 내부적 표현



이미지 데이터의 내부적 표현

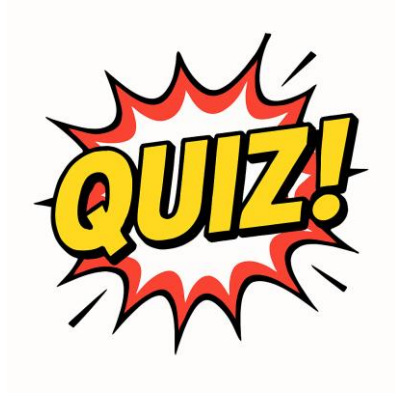


컴퓨터에게

이미지는 3차원 행렬이다.

이미지를 조작할 때

컴퓨터는 행렬 연산을 한다.

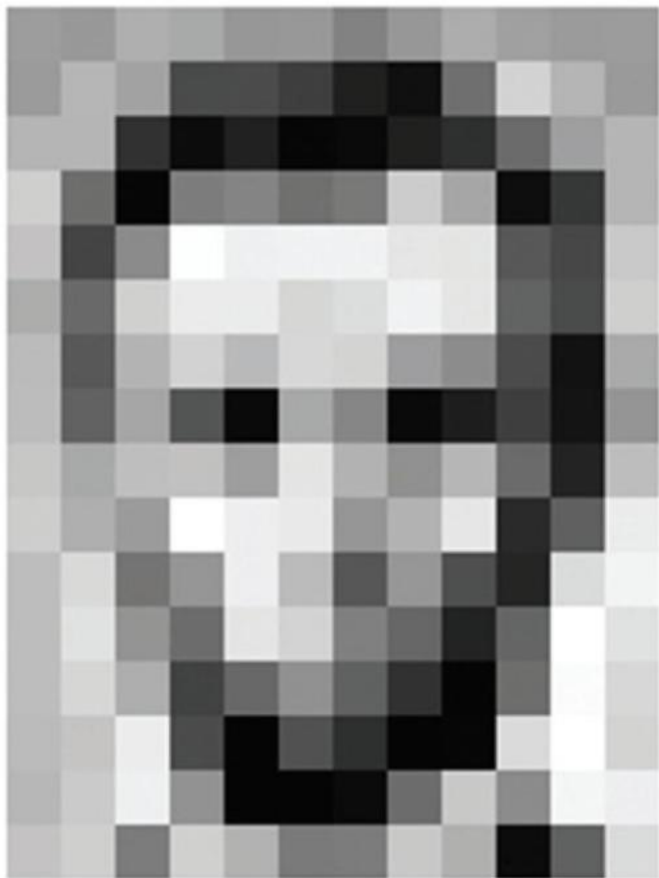


그림은 컴퓨터에게 작은 칸칸의 숫자,
즉 () 데이터로 보인다.



그림은 컴퓨터에게 작은 칸칸의 숫자,
즉 (픽셀) 데이터로 보인다.

사람의 시각 vs 컴퓨터 시각



사람의 눈 : 직관적 이해

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	106	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	106	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

컴퓨터의 눈 : 숫자 데이터 분석

픽셀만 보고 무슨 이미지인지 알 수 있을까?

컴퓨터는 픽셀 데이터에서 특징(feature)을 추출하여 이미지를 인식한다

윤곽선 개수

윤곽선 방향 분포

털 질감(texture)



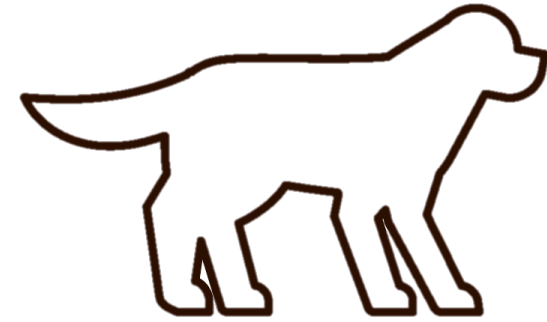
색상 히스토그램

얼굴 형태

귀 모양

대표적인 특징 : 윤곽선

- 윤곽선(Edge)
 - 이미지 데이터에서 가장 기본적이며 중요한 시각적 요소
 - 이미지에서 밝기 변화가 급격하게 일어나는 지점
 - 물체의 윤곽선이나 경계를 나타냄
 - 물체의 형태를 파악하는 데 필수적인 기초 정보



직접 윤곽선을 감지해보자

0	0	0	0	0	0	0	0	0	0	0
0	0	1	1	0	0	0	1	1	0	0
0	1	0	0	1	0	1	0	0	1	0
0	1	0	0	0	1	0	0	0	1	0
0	1	0	0	0	0	0	0	0	1	0
0	0	1	0	0	0	0	0	1	0	0
0	0	0	1	0	0	0	1	0	0	0
0	0	0	0	1	0	1	0	0	0	0
0	0	0	0	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0

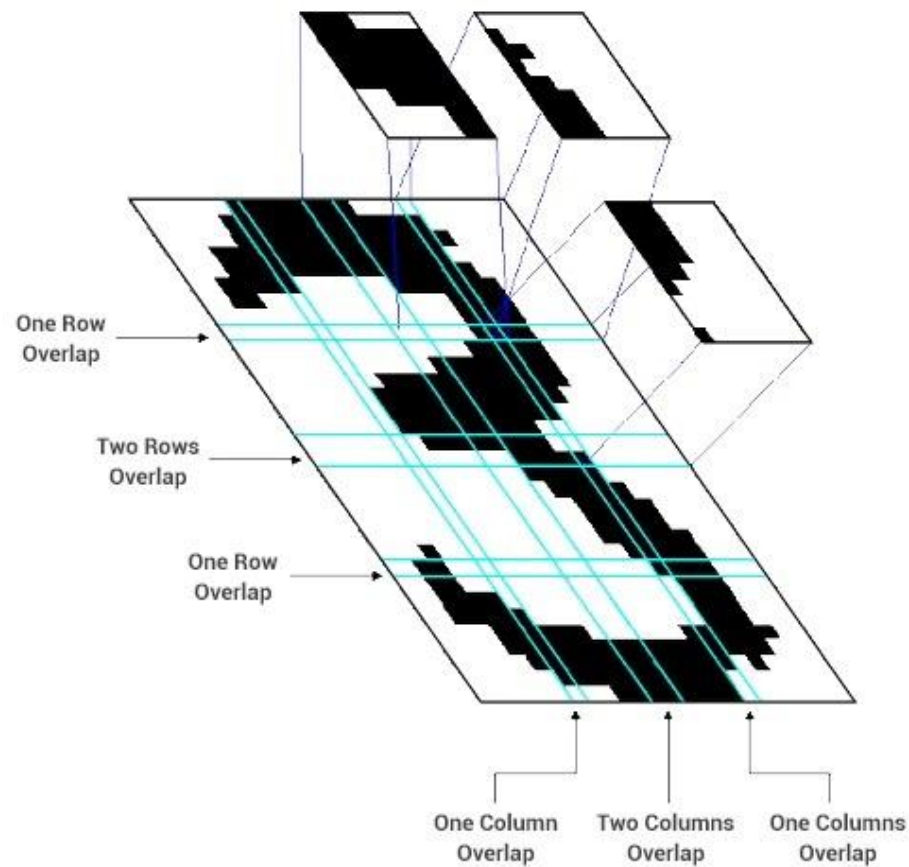
0: 흰색, 1: 검은색

21	21	21	21	21	21	21	21	21	21	21	21	21	21	21	21	21	21	21	21
21	21	21	21	21	21	21	21	21	21	21	21	21	21	21	21	21	21	21	21
21	21	21	21	21	21	21	21	21	21	21	21	21	21	21	21	21	21	21	21
21	21	21	21	21	21	21	21	21	21	21	21	21	21	21	21	21	21	21	21
21	21	21	21	21	21	21	21	21	21	145	145	21	21	21	21	21	21	21	21
21	21	21	21	21	21	21	21	21	145	145	145	145	21	21	75	75	21	21	21
21	21	21	21	21	21	21	21	145	145	145	145	145	21	75	75	21	21	21	21
21	21	21	21	21	21	21	145	145	145	145	145	145	145	75	75	21	21	21	21
21	21	21	21	21	145	145	145	145	145	145	145	145	145	75	21	21	21	21	21
21	21	21	21	145	145	145	145	145	145	145	145	145	145	145	145	21	21	21	21
21	21	21	21	21	90	90	90	90	90	90	90	90	90	90	90	21	21	21	21
21	21	21	21	21	90	90	90	90	90	90	90	90	90	90	90	21	21	21	21
21	21	21	21	21	90	90	90	90	90	90	90	90	90	90	90	21	21	21	21
21	21	21	21	21	90	90	90	90	90	90	90	90	90	90	90	21	21	21	21
21	21	21	21	21	90	90	90	90	90	90	90	90	90	90	90	21	21	21	21
21	21	21	21	21	90	90	90	90	90	90	90	90	90	90	90	21	21	21	21
21	21	21	21	21	90	90	90	90	90	90	90	90	90	90	90	21	21	21	21
21	21	21	21	21	90	90	90	90	90	90	90	90	90	90	90	21	21	21	21
21	21	21	21	21	90	90	90	90	90	90	90	90	90	90	90	21	21	21	21
21	21	21	21	21	90	90	90	90	90	90	90	90	90	90	90	21	21	21	21
21	21	21	21	21	90	90	90	90	90	90	90	90	90	90	90	21	21	21	21
21	21	21	21	21	90	90	90	90	90	90	90	90	90	90	90	21	21	21	21

0: 흰색, 1: 검은색

25

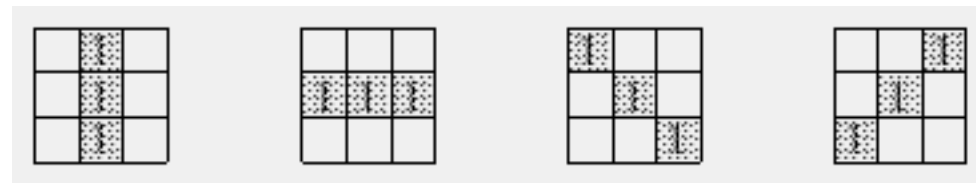
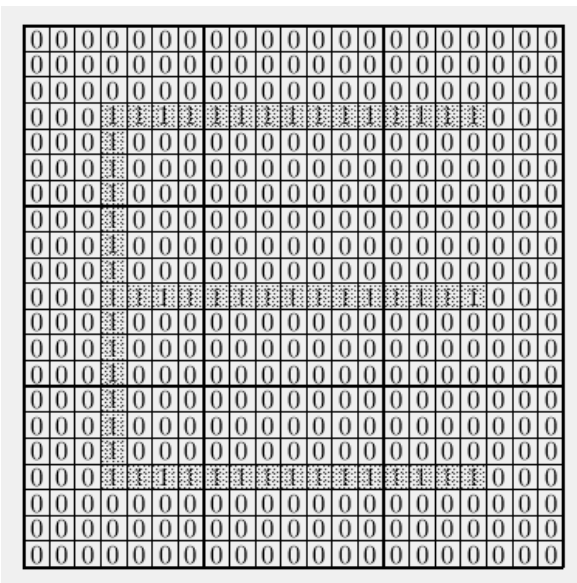
컴퓨터는 어떻게 윤곽선을 감지할까?



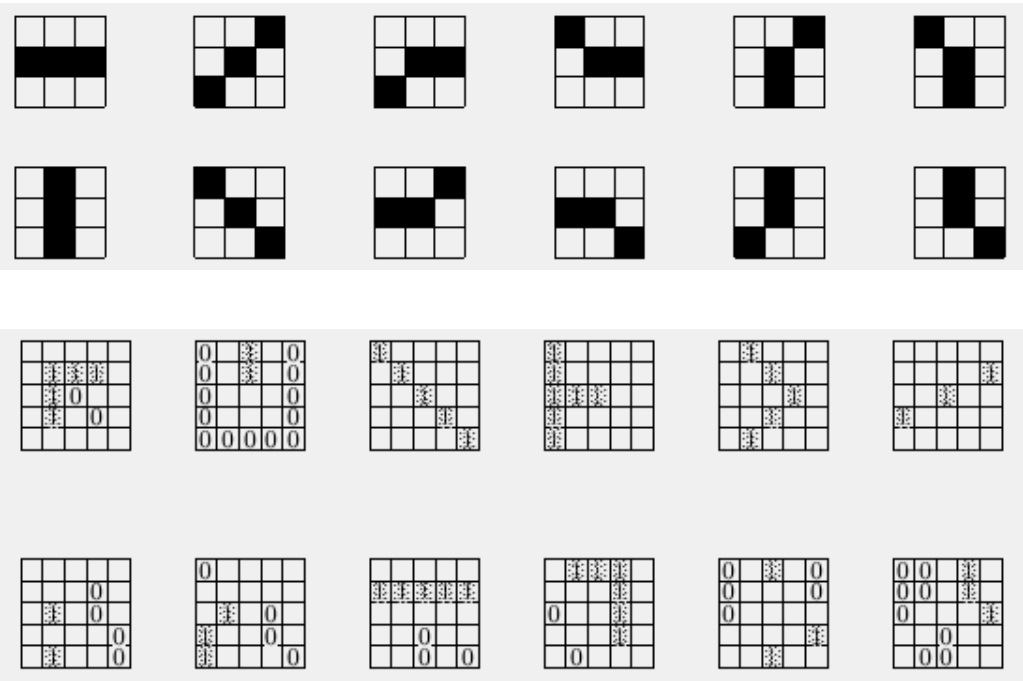
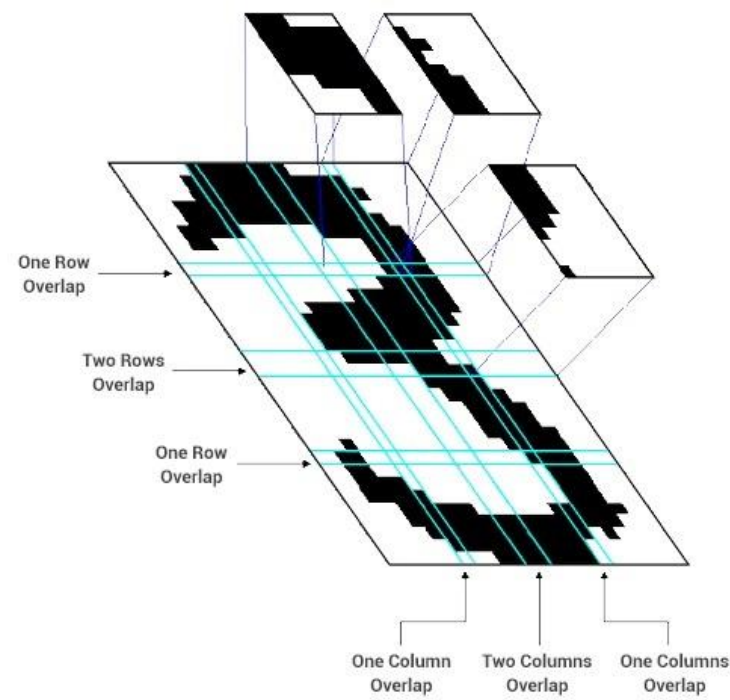


윤곽선 검출 알고리즘의 개념

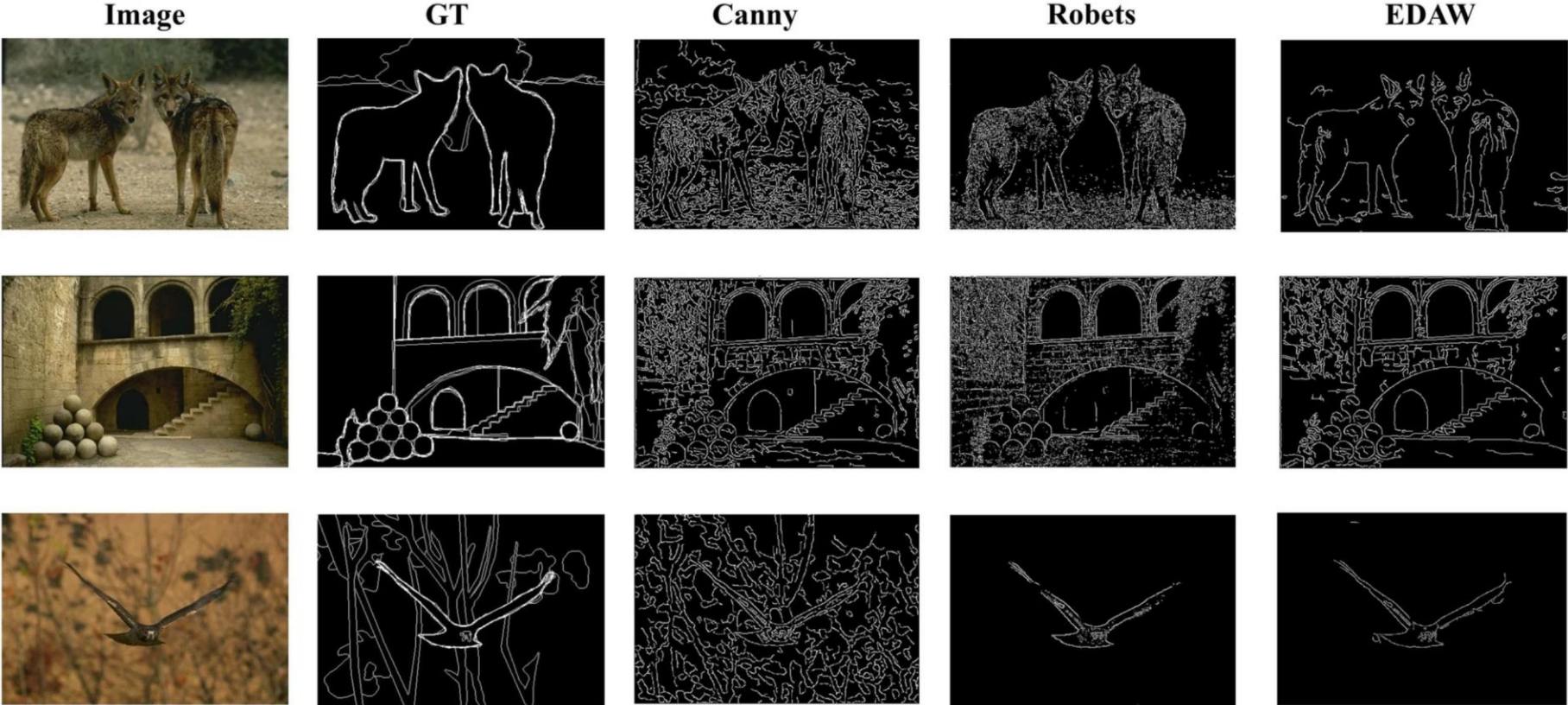
- 이미지에서 중요한 특징을 추출하기 위한 필터를 설계
- 이 필터들은 특정 패턴(윤곽선, 모서리 등)에 반응하도록 고정된 값을 가짐



유효선 검출 알고리즘의 개념



윤곽선을 검출하기 위한 다양한 알고리즘



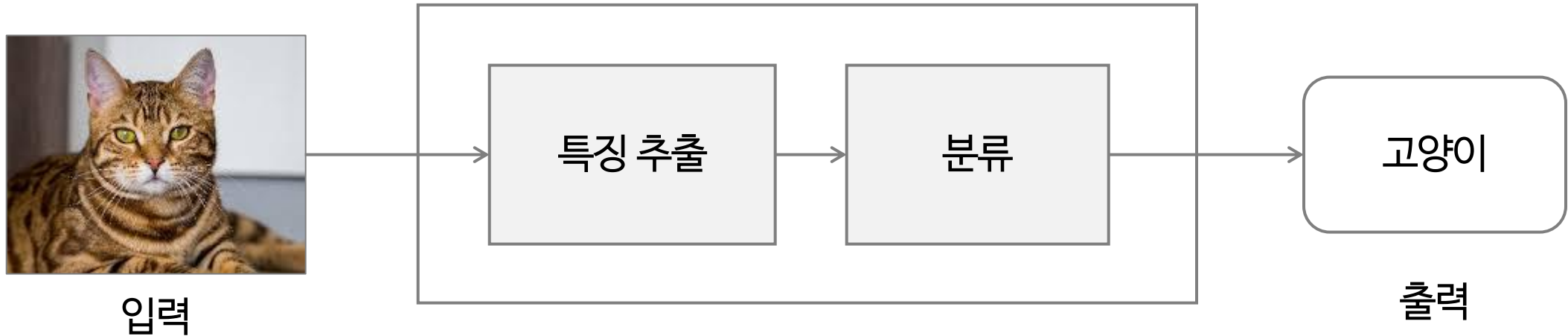
어떤 특징을 설정할 것인가?

특징을 검출하는 필터를 어떻게 설계할 것인가?

→ 이미지 인식 성능에 영향을 많이 미침

전통적 컴퓨터 비전

예) 이미지가 개인지 고양이인지를 알려주는 시스템



- 사람이 이미지를 관찰하여 특징을 결정함
- 좋은 특징을 찾는 것이 중요

가장 큰 장벽

어떤 특징이 좋은 지 사람이 알기 어렵다.

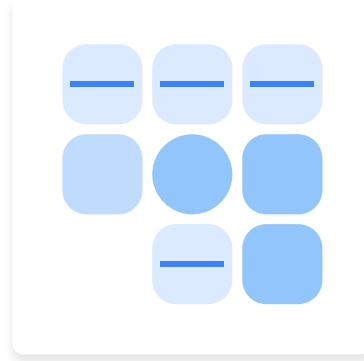
딥러닝의 이미지 인식

- 딥러닝은 특징을 사람이 만들지 않고 데이터로 학습한다.
- 어떤 특징을 볼 것인지, 각 특징을 찾는 필터까지 스스로 학습한다.



많은 사진 보기

대량의 이미지를 학습



패턴 발견하기

선, 모양, 색상 등
공통된 특징을 찾아냄



분류하기

학습한 패턴을 바탕으로
그림을 구분함

딥러닝의 이미지 인식

강아지와 고양이 구분 예시



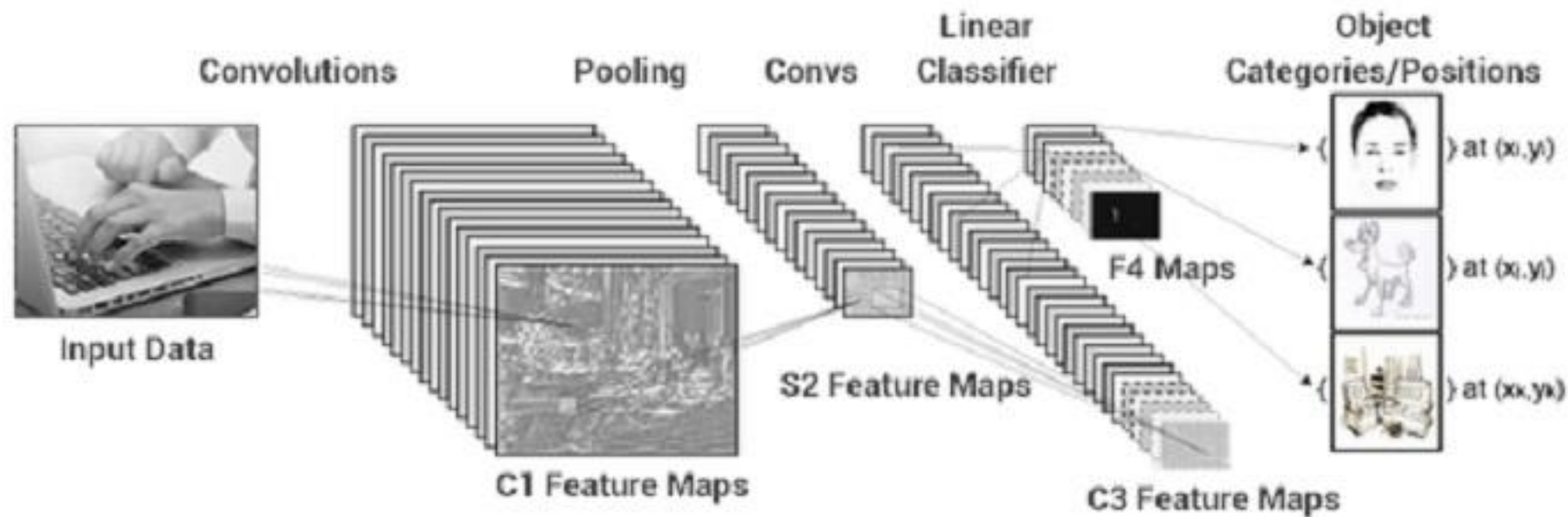
- ✓ AI는 이미지를 픽셀 단위로 분석하여 특징을 추출합니다
- ✓ 귀 모양, 코의 형태, 얼굴 윤곽, 눈의 특징 등을 비교합니다
- ✓ 머신러닝 알고리즘은 수천 장의 사진을 학습하여 패턴을 인식합니다
- ✓ 새로운 이미지가 주어졌을 때 특징점들의 조합을 통해 어떤 동물인지 구분합니다.

합성곱 신경망

CNN (Convolutional Neural Networks)

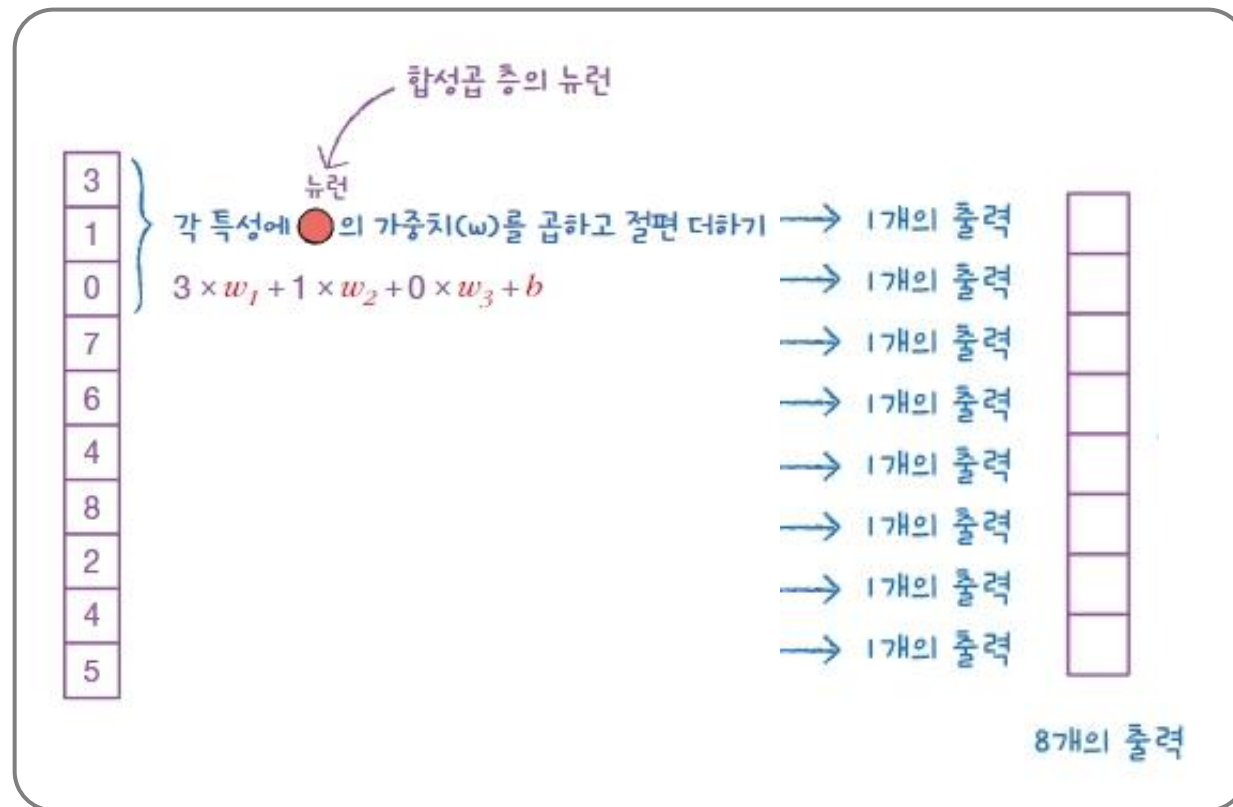
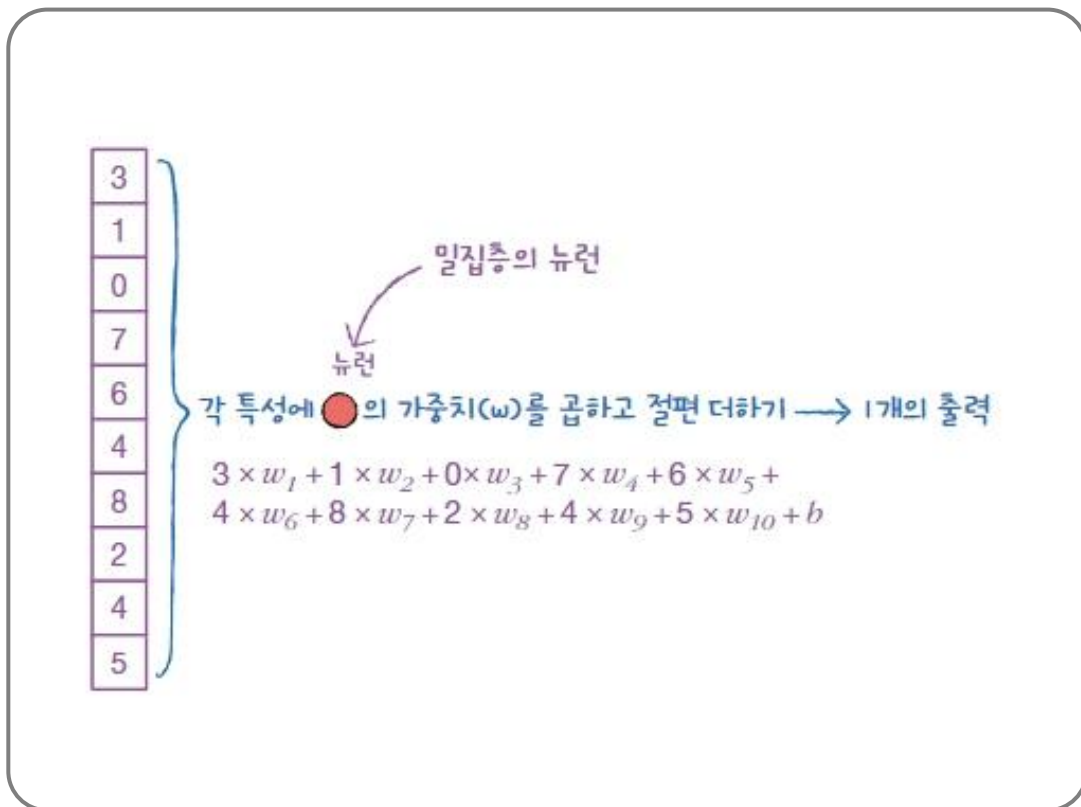
합성곱 신경망 (CNN: Convolutional Neural Network)

- 합성곱 레이어를 하나 이상 포함하는 심층 신경망



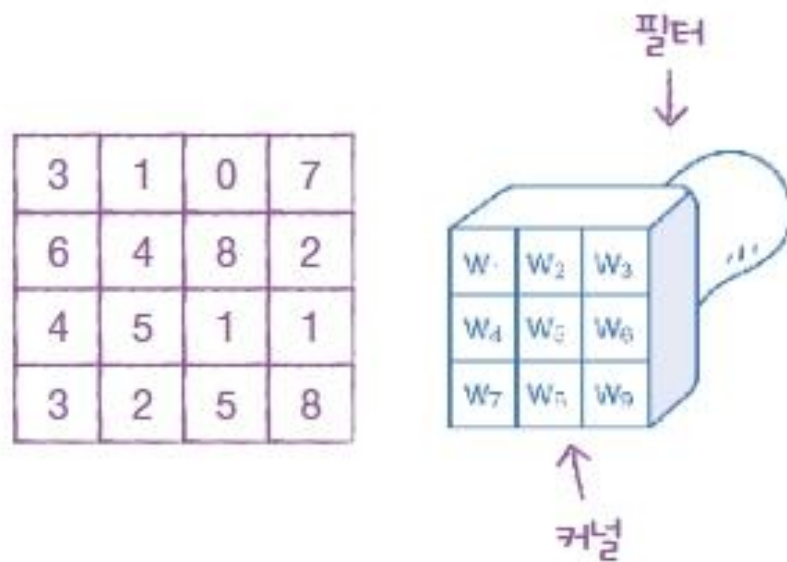
합성곱 레이어

- 1차원 데이터에서 밀집층과 합성곱 층 비교



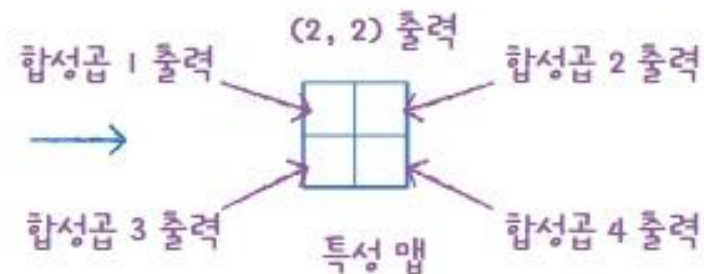
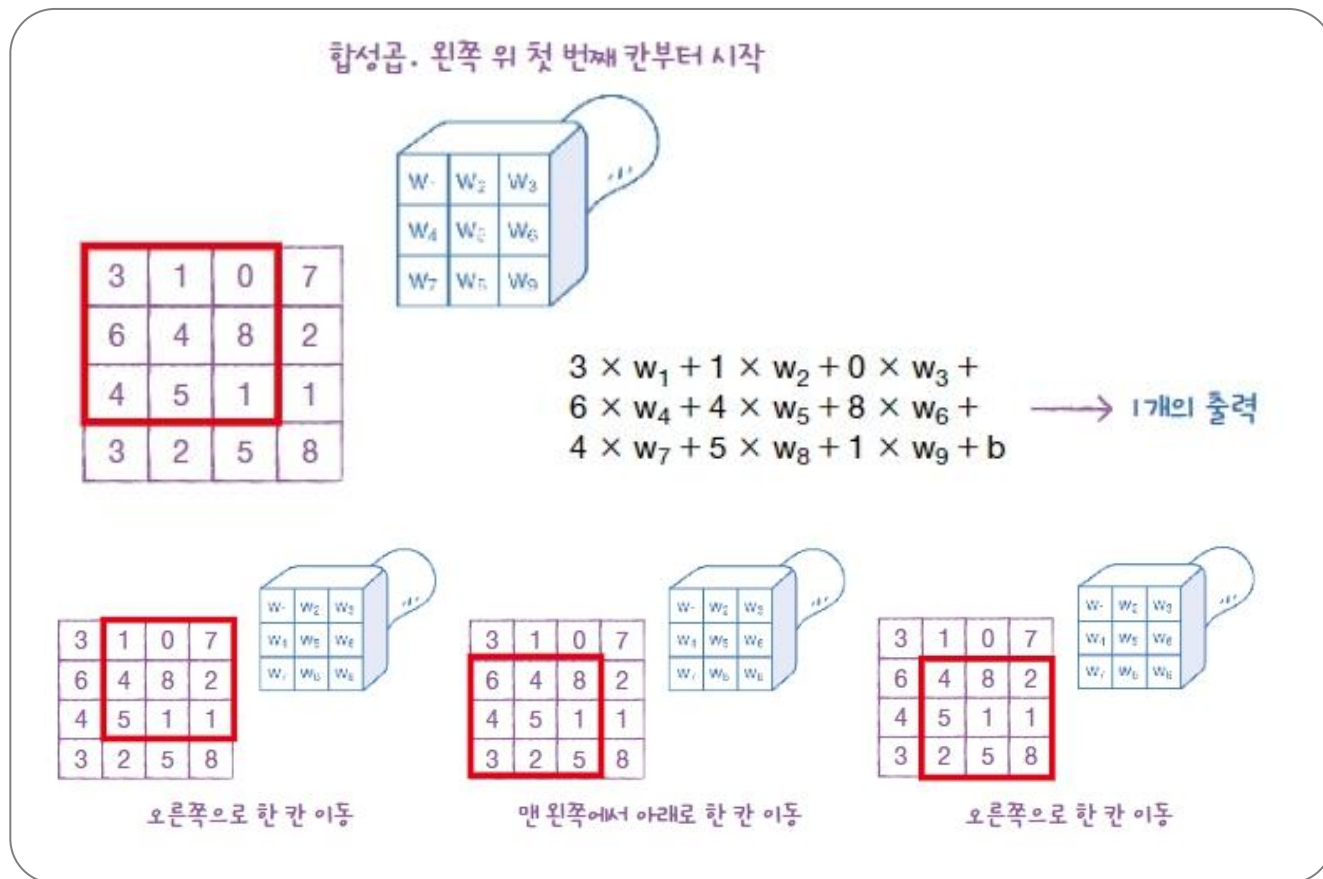
합성곱 레이어

- 용어 정리
 - 필터(filter) : CNN에서의 뉴런
 - 커널(kernel) : 필터의 가중치
- 2차원 데이터
 - 필터도 2차원이어야 함
 - 커널의 크기는 (3, 3)으로 가정



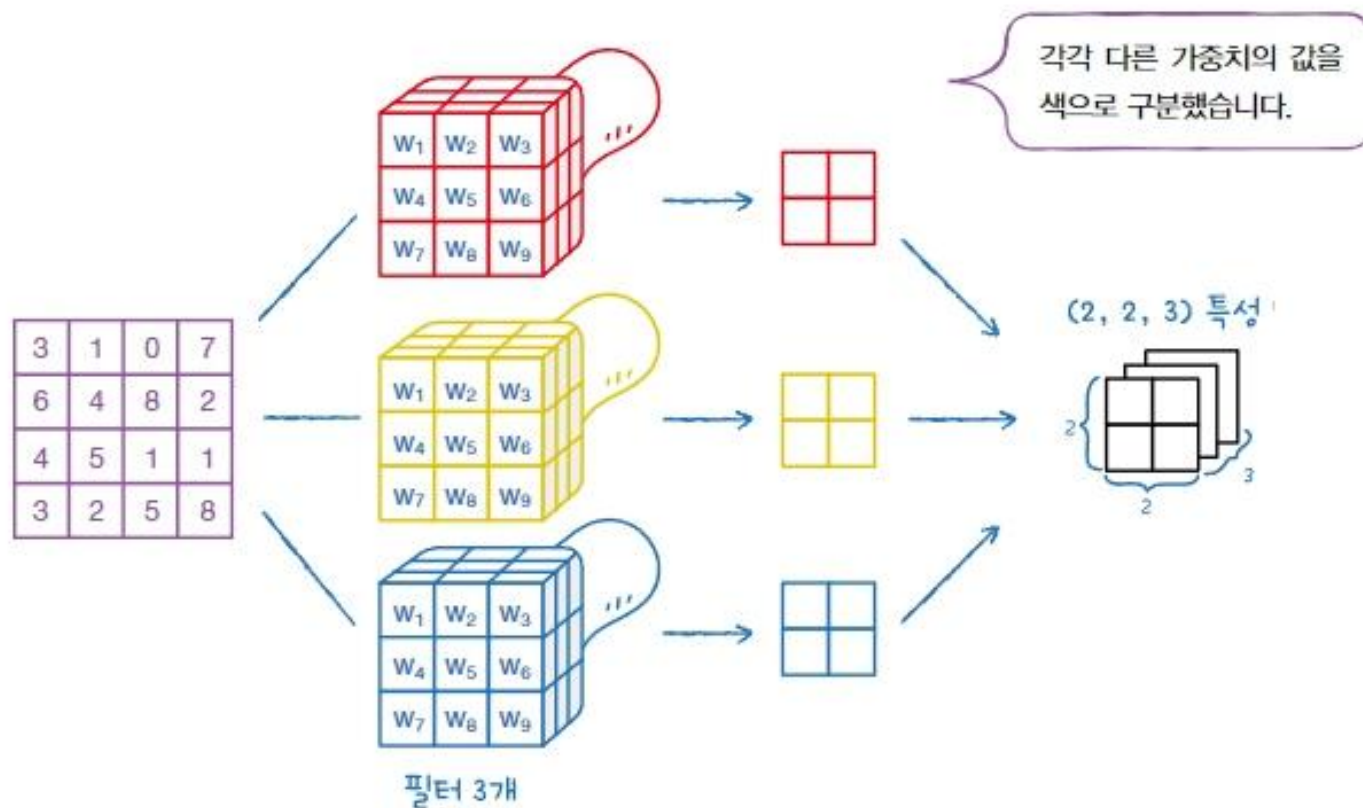
특성 맵

- 특성 맵(Feature Map): 합성곱 계산을 통해 얻은 출력



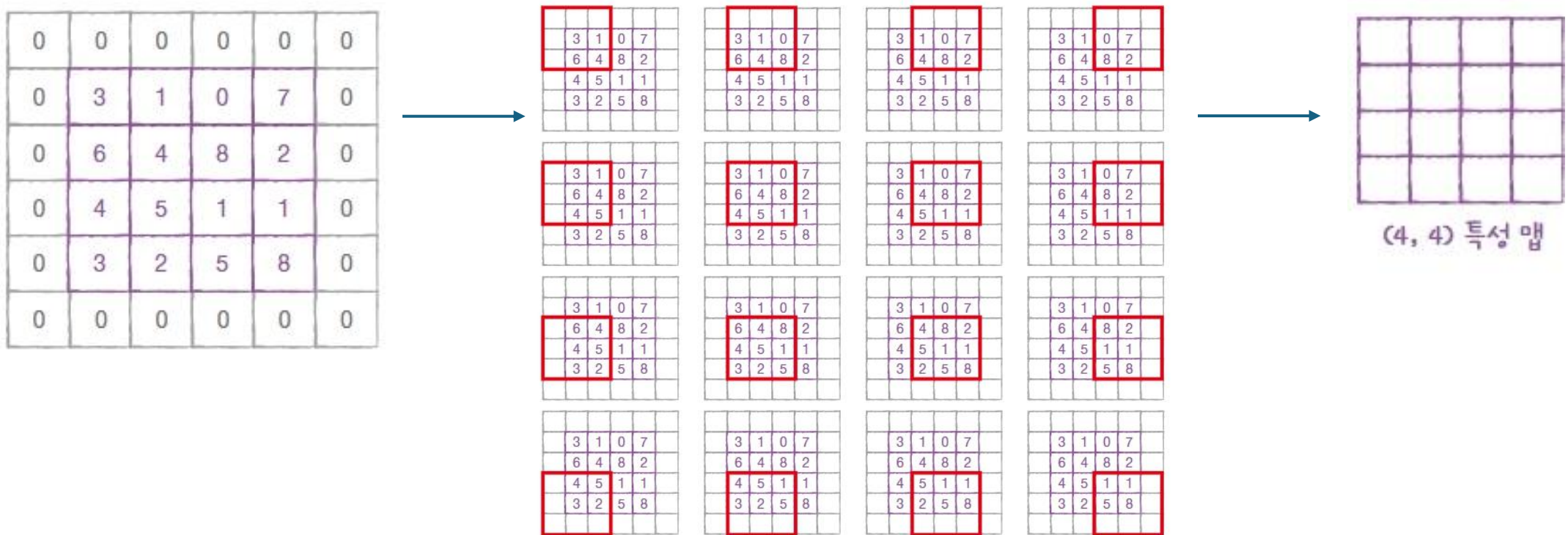
특성 맵

- 여러 개의 필터를 사용하면 특성 맵은 3차원 배열이 됨.
- 특성 맵의 깊이는 필터의 개수와 동일



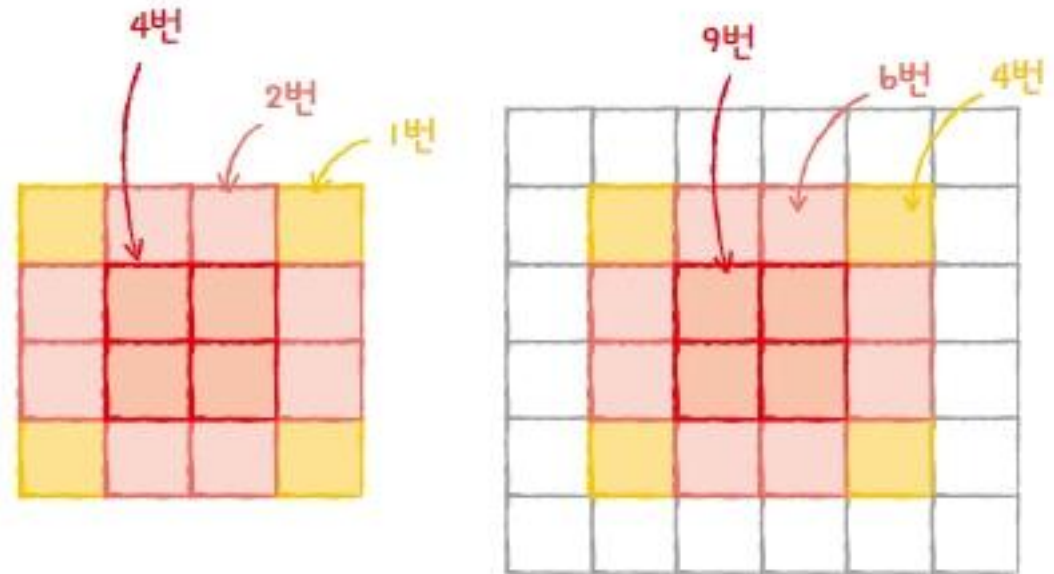
패딩

- 패딩(padding) : 입력 배열 주위를 가상의 원소로 채우는 것
 - 세임 패딩(same padding): 입력과 특성 맵의 크기를 동일하게 만들기 위해 입력 주위를 0으로 패딩
 - 밸리드 패딩(valid padding): 패딩 없이 입력으로만 특성 맵을 만드는 경우



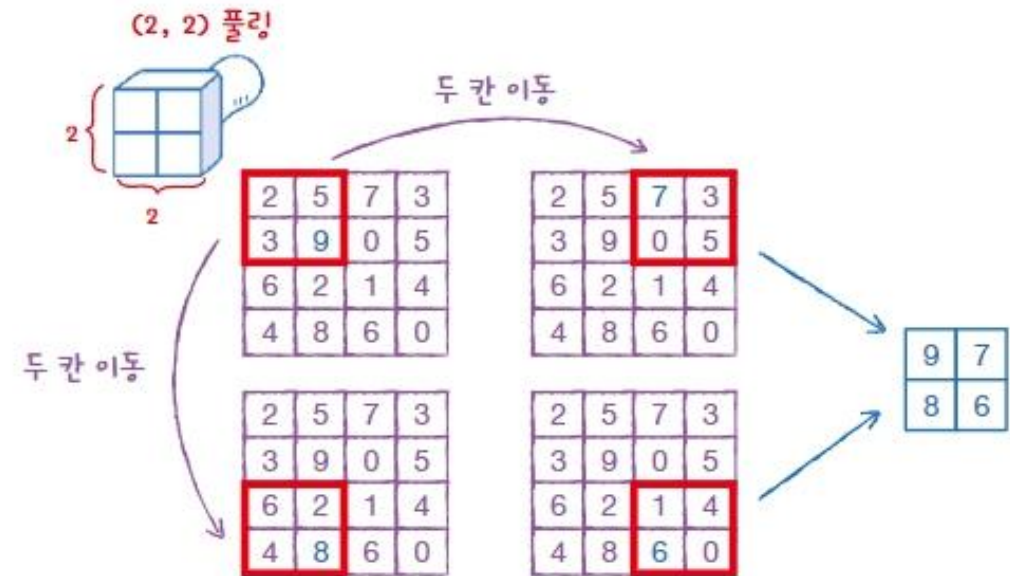
패딩

- 왜 패딩을 사용할까?
 - 입력 데이터의 위치에 따라 합성곱 계산 시 참조되는 횟수가 달라지기 때문
 - : 만약 입력이 이미지라면 모서리에 있는 중요한 정보가 특성 맵에 잘 반영되지 않을 가능성 높음
 - : 반면 가운데 있는 정보는 두드러지게 표현 됨
 - 1픽셀 패딩 → 참조 횟수 차이 완화
 - 2픽셀 패딩 → 참조 횟수 동일해 짐



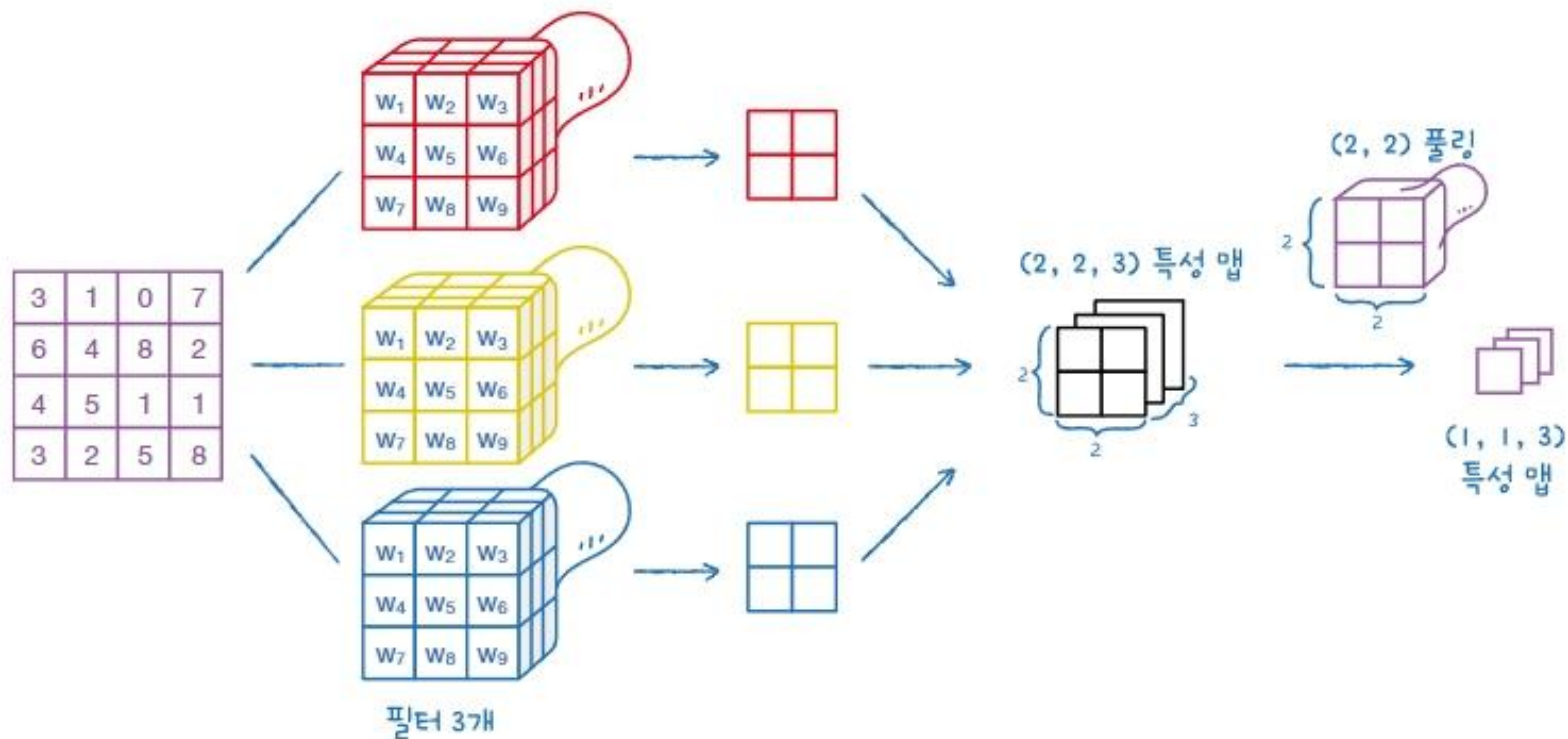
풀링

- 풀링(pooling): 특성 맵의 가로세로 크기를 줄이는 것
 - 풀링에는 가중치가 없음
 - 풀링 영역 안에서 최대 값 또는 평균을 계산함 → 각각 **최대 풀링**, **평균 풀링**이라고 부름
 - 데이터가 겹치지 않게 이동함



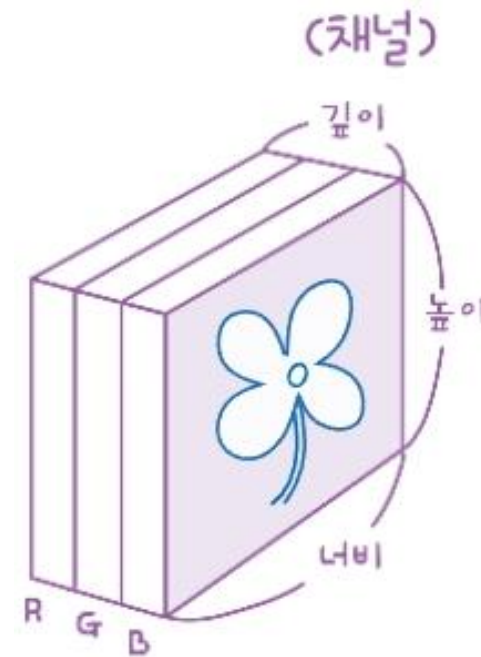
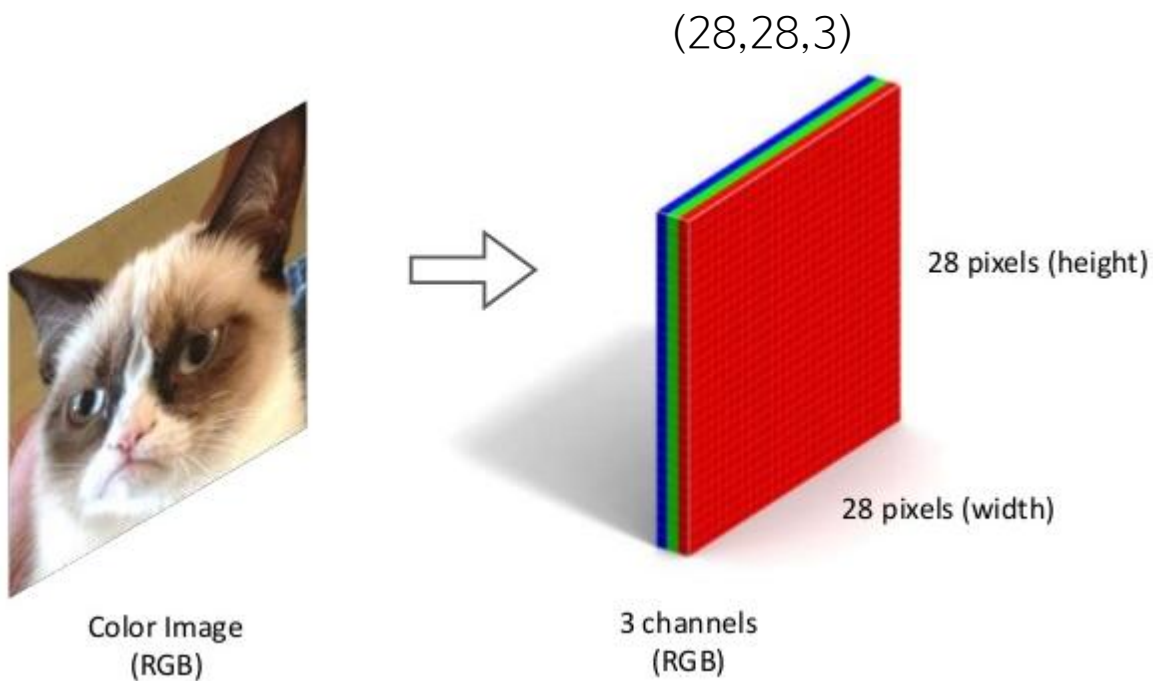
(2, 2) 크기로 최대 풀링 예시

- 풀링의 출력
 - 풀링의 출력도 특성 맵으로 봄
 - 가로세로는 줄어들지만 깊이는 변하지 않음 (*깊이* == 이전 CNN 층의 필터 개수)



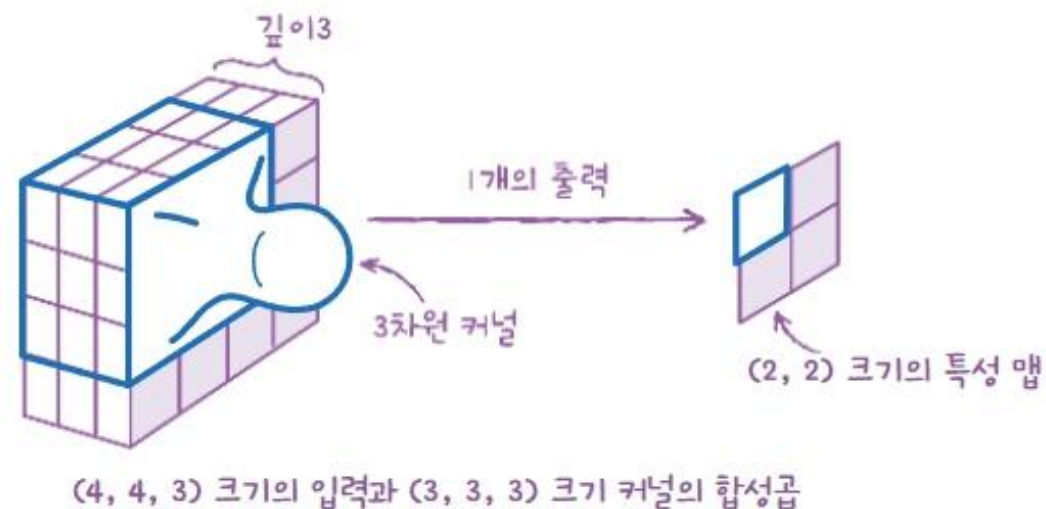
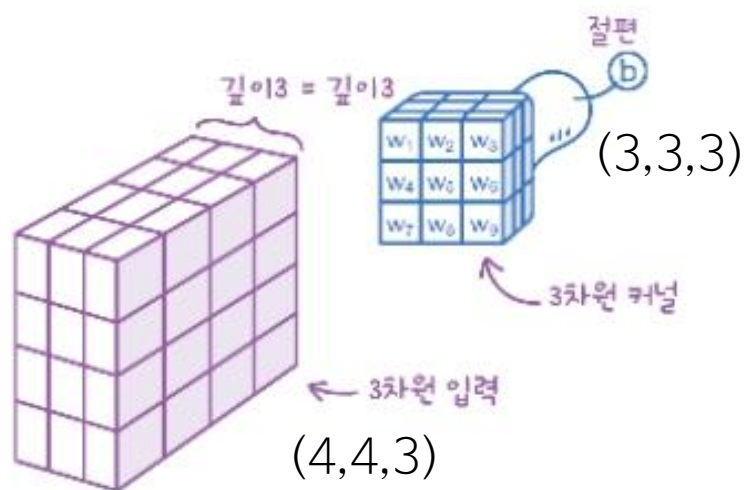
컬러 이미지의 합성곱

- 컬러 이미지는 RGB 채널로 구성되어 있기 때문에 3차원 배열로 표시함



컬러 이미지의 합성곱

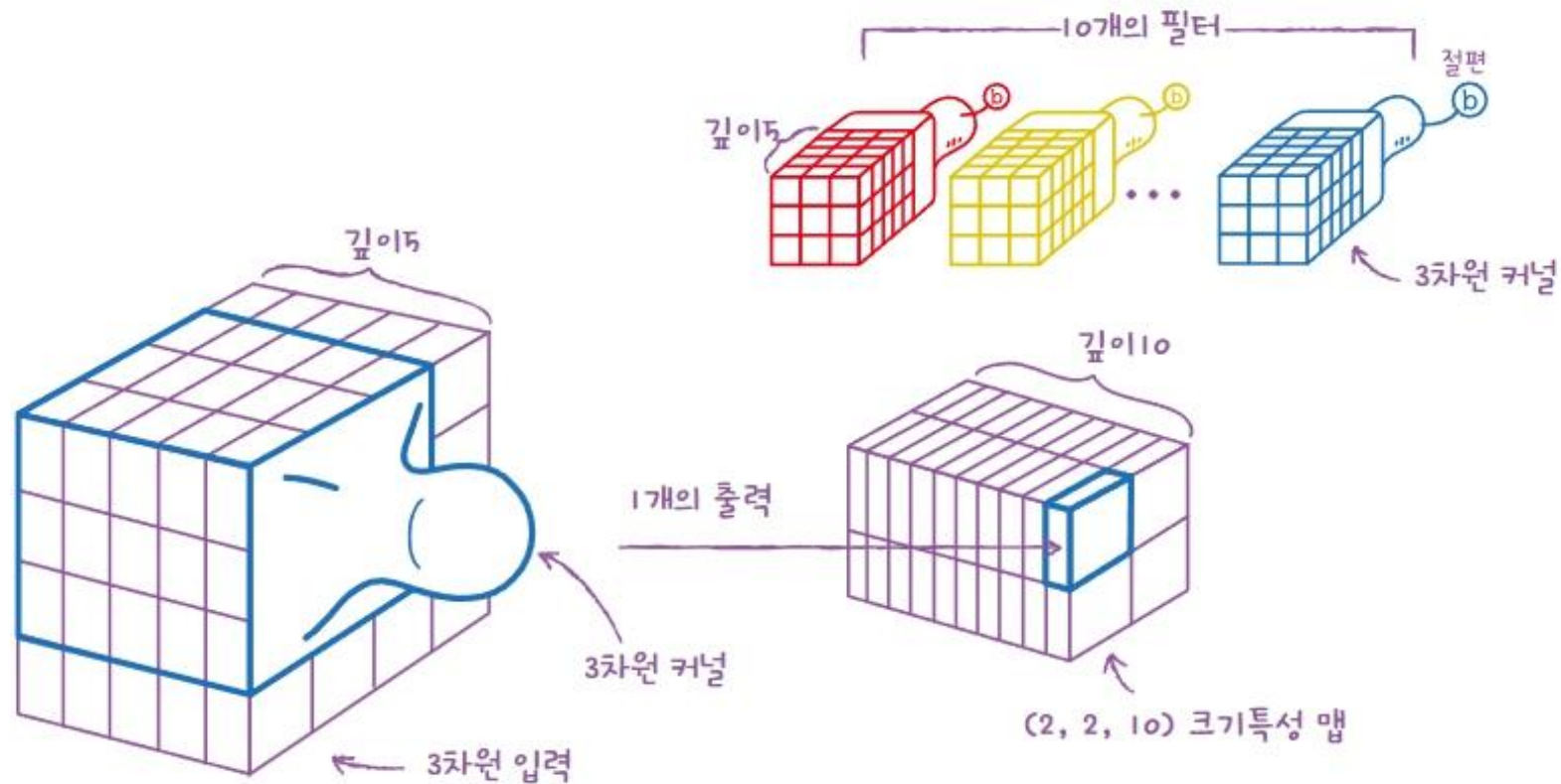
- 컬러 이미지의 경우, **입력의 깊이와 동일한 깊이를 갖는 필터가 필요함**



- 합성곱 계산 = $(3, 3, 3)$ 영역에 해당하는 27개 원소에 27개의 가중치를 곱하고 절편을 더함
- 깊이에 상관없이 한 번의 합성곱 계산의 출력은 항상 하나. 즉 특성 맵의 요소 하나가 채워 짐

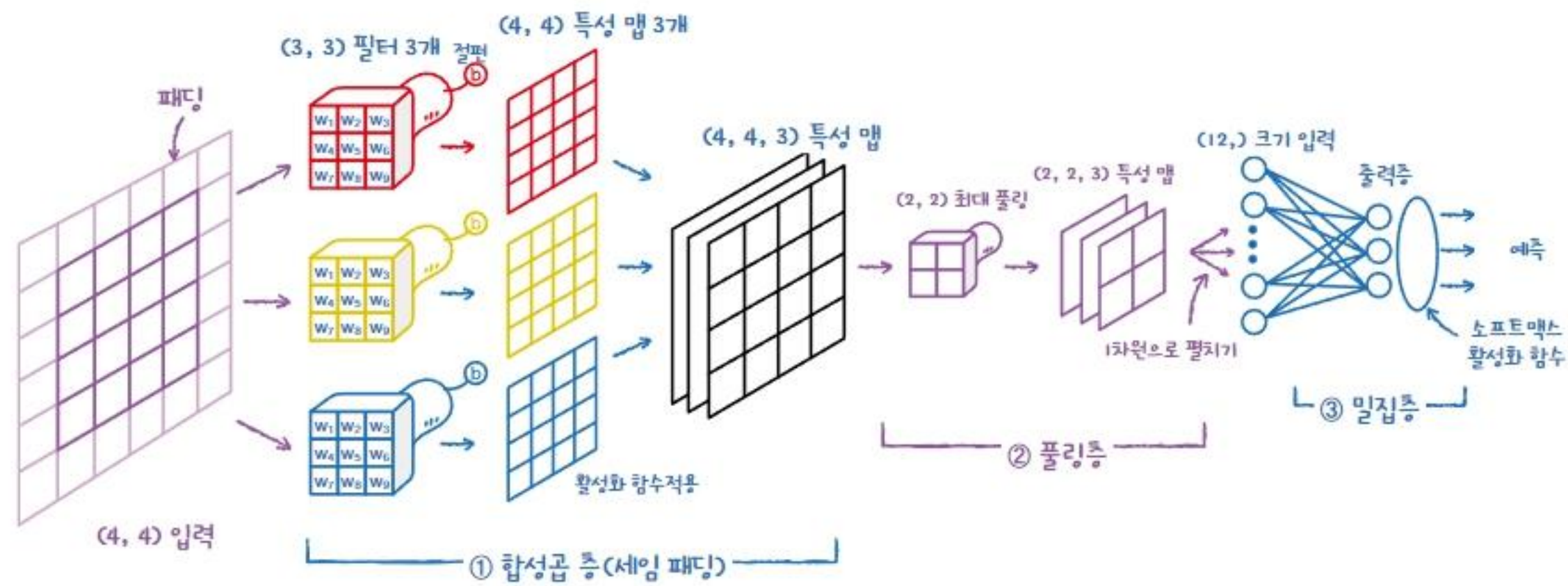
컬러 이미지의 합성곱

- 특성 맵의 깊이는 필터의 개수에 영향을 받는 값

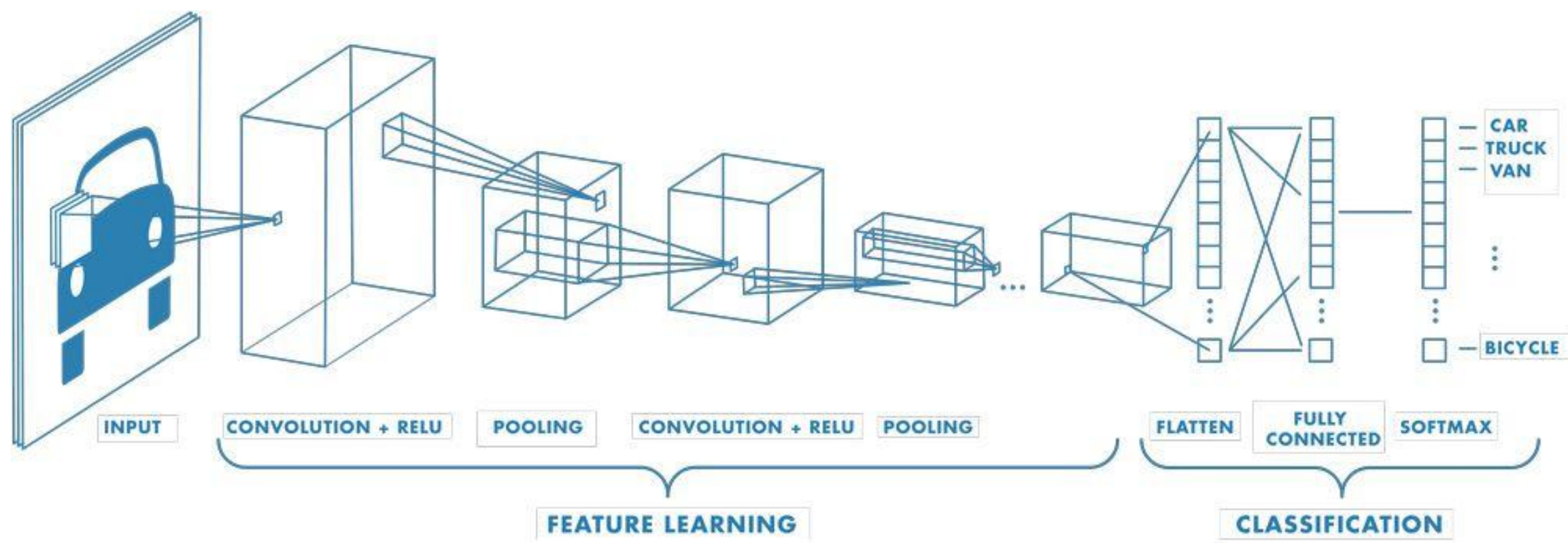


(4, 4, 5) 크기 입력과 (3, 3, 5) 크기 커널의 합성곱

합성곱 신경망 전체 구조



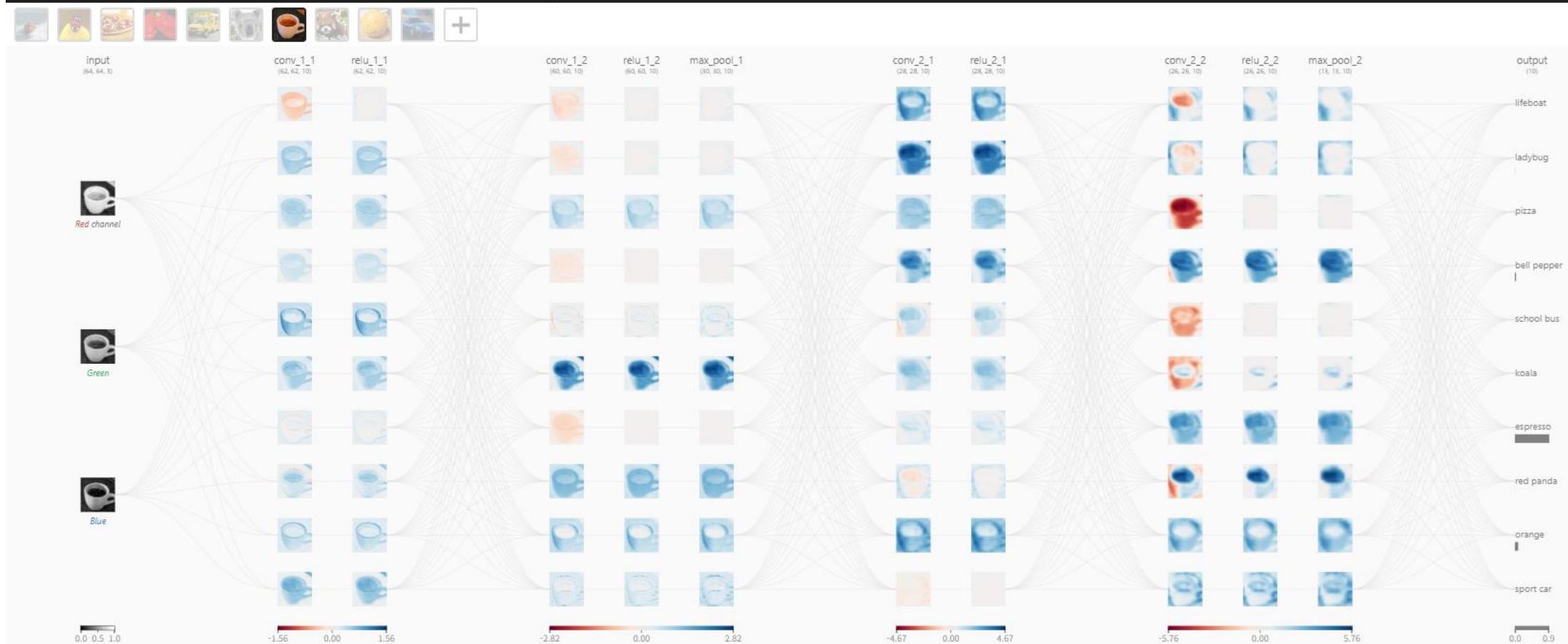
합성곱 신경망 전체 구조



CNN Explainer : 합성곱 신경망의 시각화를 볼 수 있는 사이트

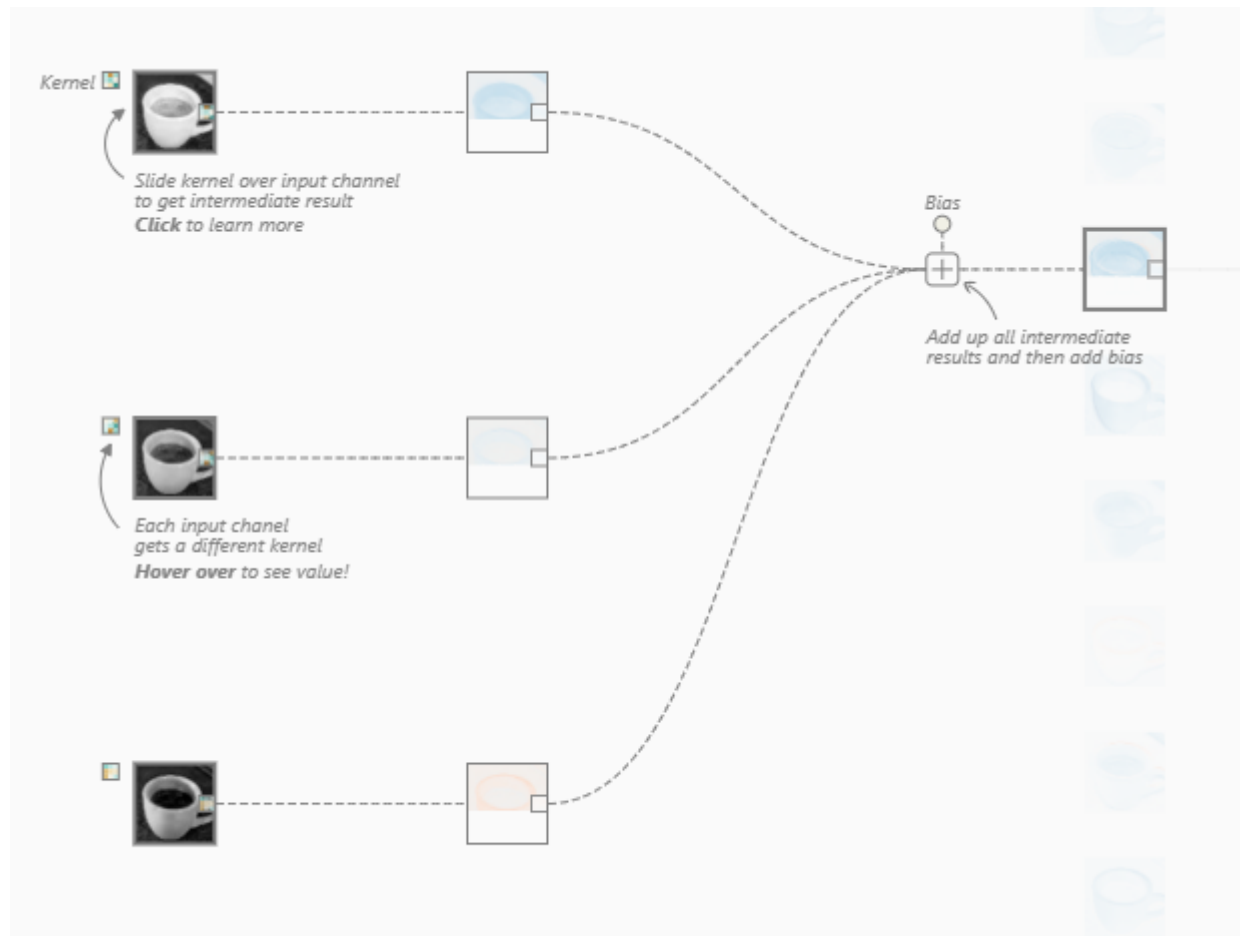
<https://poloclub.github.io/cnn-explainer/>

CNN EXPLAINER Learn Convolutional Neural Network (CNN) in your browser!



CNN Explainer : 합성곱 신경망의 시각화를 볼 수 있는 사이트

<https://poloclub.github.io/cnn-explainer/>



실습1. 손글씨 숫자 분류기

- 실습 내용: 손글씨로 쓰인 숫자 이미지 분류하기

- 실습 단계:

- ① 데이터셋 로딩
- ② 데이터 전처리
- ③ 신경망 모델 설계
- ④ 모델 컴파일
- ⑤ 모델 학습
- ⑥ 모델 평가

실습1. 손글씨 숫자 분류기

- 실습 데이터 : Keras에서 제공하는 MNIST



① 데이터셋 로딩

- Keras에서 제공하는 datasets 중 MNIST 데이터셋을 `load_data()` 함수로 불러온다.
- 불러온 데이터의 shape을 확인한다.

```
import tensorflow as tf
from tensorflow import keras
import numpy as np

# 1. 데이터셋 불러오기
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
```

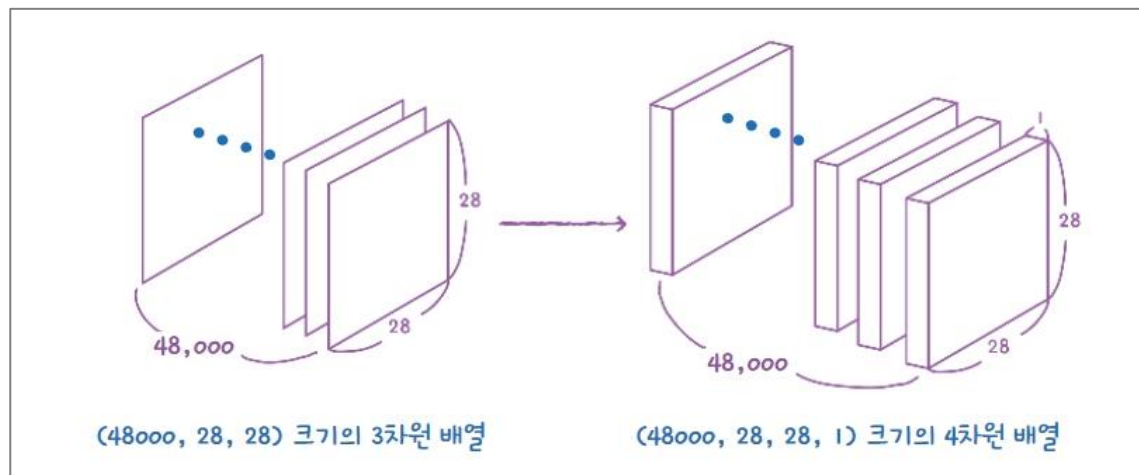
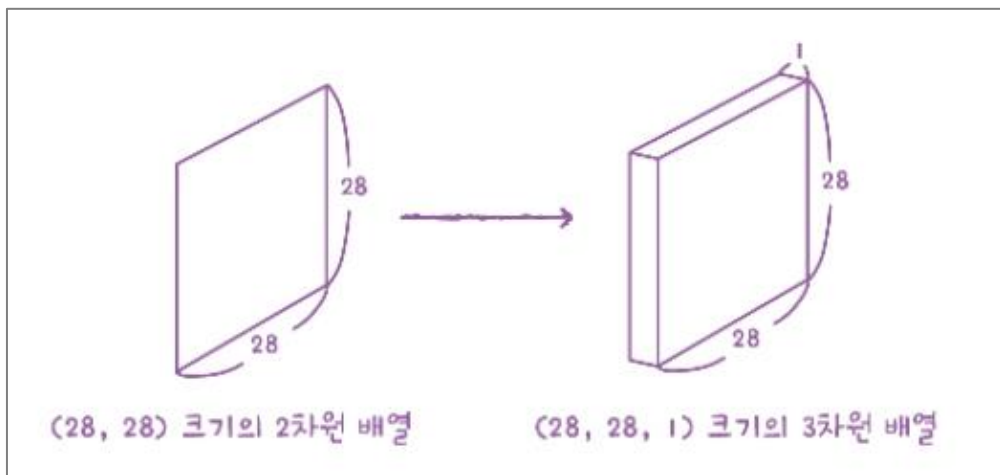
```
print(type(x_train))
print(x_train.shape)
print(x_test.shape)

<class 'numpy.ndarray'>
(60000, 28, 28)
(10000, 28, 28)
```

※ [주의] : `train_test_split()` 함수와 반환값의 순서가 다름!

② 데이터 전처리

- 케라스는 합성곱 레이어의 입력으로 3차원 데이터를 기대 함
- 흑백 이미지 이미지인 경우, 깊이를 한 층 추가해서 3차원으로 변환해야 함
- 데이터의 개수까지 더해지면 입력 데이터의 shape은 4차원이 됨



② 데이터 전처리

- (28, 28)인 흑백 이미지를 (28, 28, 1)로 변환

```
# 2. 데이터 전처리
# (28, 28) 이미지를 (28, 28, 1) 형태로 변환 (채널 1개: 흑백)
X_train = X_train.reshape((60000, 28, 28, 1))
X_test = X_test.reshape((10000, 28, 28, 1))

# 결과 확인
print(X_train.shape)
print(X_test.shape)
```

```
(60000, 28, 28, 1)
(10000, 28, 28, 1)
```

- 흑백 이미지는 각 픽셀마다 0~255 값을 가진다. 255로 나누어 0~1 사이의 값을 갖도록 정규화

```
# 데이터 정규화
X_train = X_train / 255.0
X_test = X_test / 255.0
```

③ 신경망 모델 설계

- 신경망 모델 생성하는 전체 코드

```
# 3. 모델 구축 (가장 간단한 CNN)
model = keras.Sequential([
    # 입력 레이어: 학습 데이터의 shape 설정
    keras.layers.Input(shape=(28, 28, 1)),

    # 합성곱 레이어: 32개의 3x3 필터
    keras.layers.Conv2D(32, (3, 3), activation='relu'),
    # 맥스 풀링 레이어: 2x2 영역에서 가장 큰 값 추출
    keras.layers.MaxPooling2D((2, 2)),

    # 평탄화 : 데이터를 1차원으로 펼치기
    keras.layers.Flatten(),
    # 완전 연결 레이어: 10개의 뉴런 (0-9까지의 숫자 분류)
    keras.layers.Dense(10, activation='softmax')
])
```

③ 신경망 모델 설계

```
# 첫 번째 합성곱 레이어: 32개의 3x3 필터  
keras.layers.Conv2D(32, (3, 3), activation='relu'),  
# 맥스 풀링 레이어: 2x2 영역에서 가장 큰 값 추출  
keras.layers.MaxPooling2D((2, 2)),
```

- Conv2D : 2차원 합성곱 레이어를 만드는 클래스

- : 이미지에서 특성을 추출. 필터를 사용하여 이미지의 특정 영역을 훑어가며 패턴을 감지함.
- : **filters**: 필터(filter)의 개수. 이 값은 이 레이어에서 생성될 특징 맵(feature map)의 개수와 같음.
 - 예를 들어, 32로 설정하면 32개의 각기 다른 특징(가로선, 세로선 등)을 학습하게 됨
- : **kernel_size**: 필터의 크기를 지정. 일반적으로 정사각형 형태인 (3, 3)이나 (5, 5)와 같이 튜플 형태로 입력
- : **activation**: 합성곱 연산 후 적용할 활성화 함수를 지정. 주로 ReLU를 사용

③ 신경망 모델 설계

```
# 첫 번째 합성곱 레이어: 32개의 3x3 필터
keras.layers.Conv2D(32, (3, 3), activation='relu'),
# 맥스 풀링 레이어: 2x2 영역에서 가장 큰 값 추출
keras.layers.MaxPooling2D((2, 2)),
```

- **MaxPooling2D**: 2차원 최대 풀링(Max Pooling) 레이어를 만드는 클래스
 - : 합성곱 레이어의 결과로 나온 특징 맵(Feature Map)의 크기를 줄여주는 역할.
 - : pool_size로 지정된 영역 안에서 가장 큰 값 하나만 남기고 나머지는 버리는 방식으로 작동
 - 예를 들어, (2, 2) 풀링을 적용하면, 2x2 크기의 픽셀 4개 중에서 가장 큰 값 하나만 선택하여 남김
 - : pool_size: 최댓값을 추출할 영역의 크기를 지정. 일반적으로 (2, 2)를 사용.

③ 신경망 모델 설계

- 각 레이어에서 다음을 계산할 수 있어야 한다.
 - : 입력의 Shape과 출력의 Shape
 - : 모델이 학습하는 파라미터의 개수

모델 구조 요약

```
model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_1 (MaxPooling2D)	(None, 13, 13, 32)	0
flatten_1 (Flatten)	(None, 5408)	0
dense_1 (Dense)	(None, 10)	54,090

Total params: 54,410 (212.54 KB)

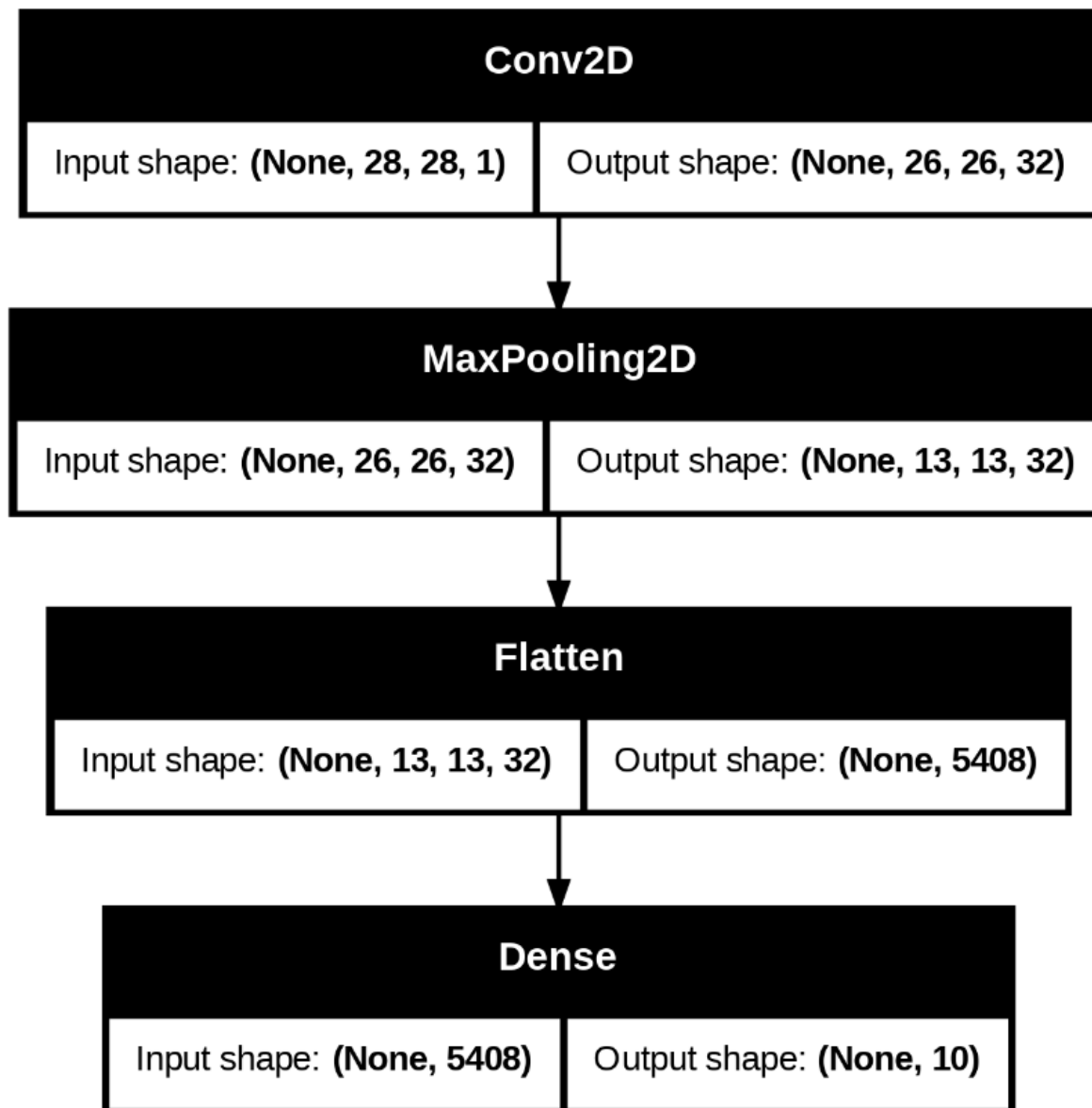
Trainable params: 54,410 (212.54 KB)

Non-trainable params: 0 (0.00 B)

③ 신경망 모델 설계

모델 구조를 이미지로 확인

```
keras.utils.plot_model(model, show_shapes=True)
```



④ 모델 컴파일

- CNN 모델을 학습시키기 위한 학습과정을 설정하는 단계
- 손실함수(loss), 최적화 로직(optimizer), 학습 중 기록할 평가 항목(metrics)을 지정
- 다중 분류이므로 손실함수로 sparse_categorical_crossentropy 설정

```
# 4. 모델 컴파일
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

⑤ 모델 학습

5. 모델 학습

```
history = model.fit(X_train, y_train,  
                    epochs=5,  
                    batch_size=64,  
                    validation_split=0.2)
```

Epoch 1/5

750/750 ————— 22s 29ms/step - accuracy: 0.8450 - loss: 0.5965 - val_accuracy: 0.9641 - val_loss: 0.1300

Epoch 2/5

750/750 ————— 41s 29ms/step - accuracy: 0.9681 - loss: 0.1136 - val_accuracy: 0.9758 - val_loss: 0.0888

Epoch 3/5

750/750 ————— 41s 29ms/step - accuracy: 0.9796 - loss: 0.0720 - val_accuracy: 0.9780 - val_loss: 0.0757

Epoch 4/5

750/750 ————— 41s 29ms/step - accuracy: 0.9842 - loss: 0.0550 - val_accuracy: 0.9801 - val_loss: 0.0706

Epoch 5/5

750/750 ————— 41s 29ms/step - accuracy: 0.9857 - loss: 0.0494 - val_accuracy: 0.9815 - val_loss: 0.0638

⑥ 모델 평가

6. 모델 평가

```
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=2)
print(f'\n테스트 정확도: {test_acc:.4f}')
```

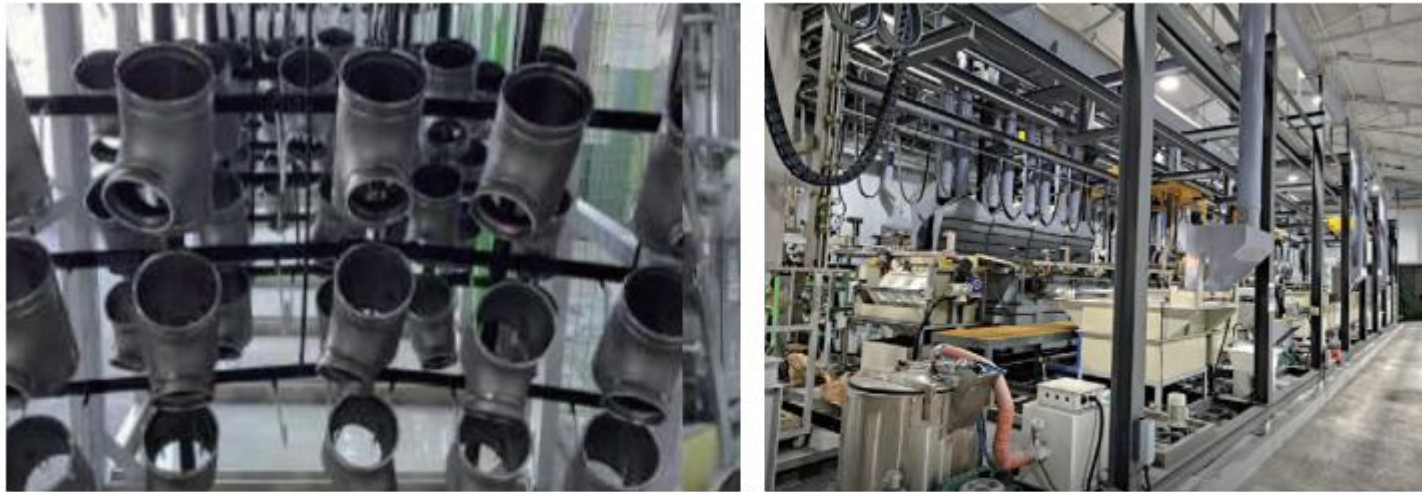
```
313/313 - 3s - 8ms/step - accuracy: 0.9829 - loss: 0.0548
```

```
테스트 정확도: 0.9829
```

이미지 제조데이터 기반 AI 개발

전기아연도금 제조공장

- 울산광역시에 있는 K사는 금속제품에 대해 도금처리를 하는 표면처리 전문기업
- 납품하기 전 흠집, 도금 미흡, 이물질 발생 등 불량품을 육안으로 검사하였으나 생산성 저하를 개선하기 위해 AI를 도입하고자 함



<출처 : 인공지능 제조플랫폼(KAMP)>

데이터셋 스크립트



<정상 제조데이터>



<불량 제조데이터>

헤더		바디			
No	변수명	척도	유형 (타입)	변수 설명	속성/범주값
1	정상	정상 이미지	.png	도금공정에서 정상판 정 받은 도금체 이미지 제조데이터	
2	불량	불량 이미지	.png	도금공정에서 불량판 정 받은 도금체 이미지 제조데이터	

실습2. 이미지 기반 제조데이터의 AI 개발

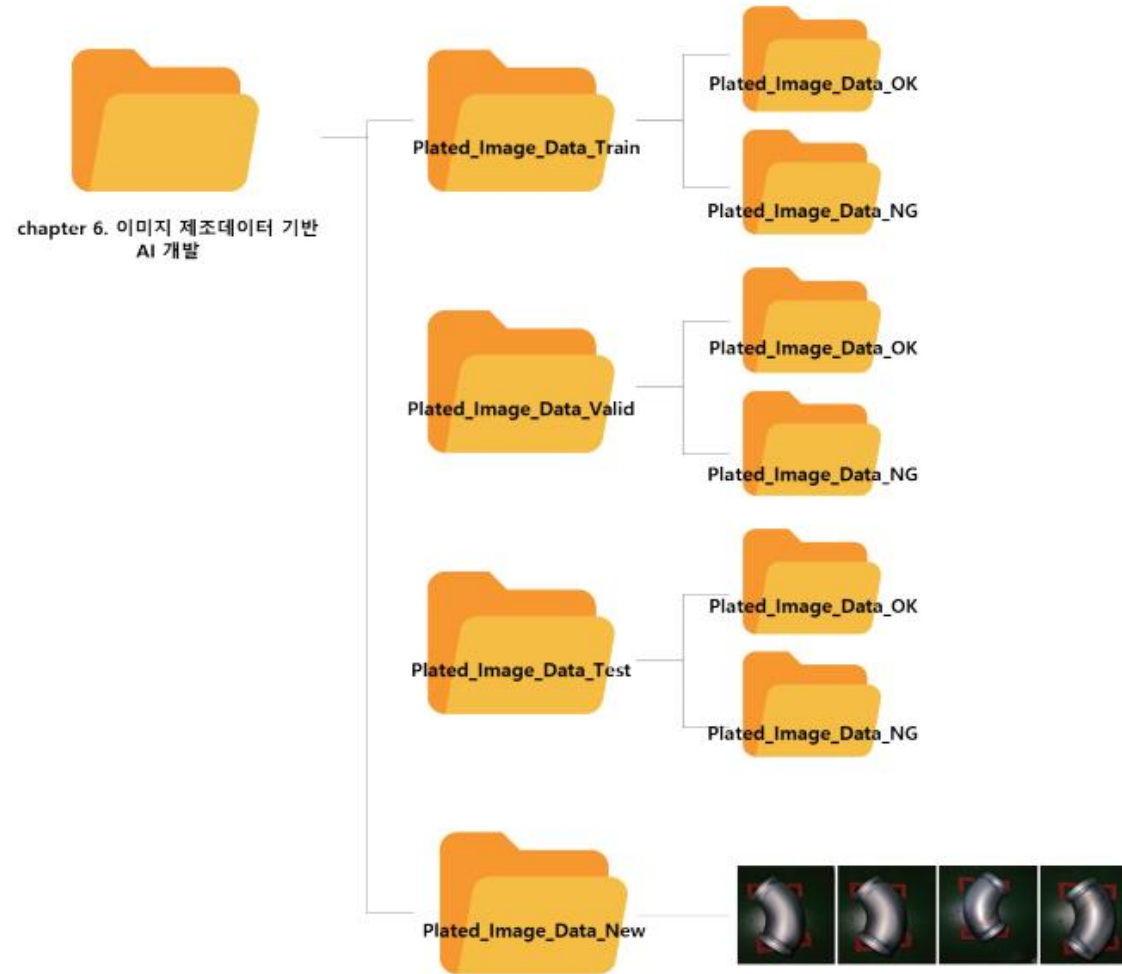
- 실습 내용: 도금 처리된 제품 이미지 데이터로부터 정상/불량 판별하기

- 실습 단계 :

- ① 데이터셋 로딩
- ② 데이터 전처리
- ③ 모델링
- ④ 학습 결과 확인
- ⑤ 새로운 데이터에 대한 예측 구하기

① 데이터셋 로딩

- 데이터 폴더 구조



② 데이터 전처리

- ImageDataGenerator

- 대량의 이미지 데이터를 효율적으로 준비하고 다양화 시켜줌
- 이미지 증강
 - : 회전, 이동, 확대/축소, 좌우반전, 밝기 조절
- 이미지 전처리
 - : 스케일링, 데이터 형식 변환
- 배치 단위 데이터 생성

② 데이터 전처리

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# train data 전처리 옵션 설정
train_datagen = ImageDataGenerator(rescale = 1./255,
                                   rotation_range = 40,
                                   width_shift_range = 0.2,
                                   height_shift_range = 0.2,
                                   shear_range = 0.2,
                                   zoom_range = 0.2,
                                   horizontal_flip = True,
                                   vertical_flip = True,
                                   fill_mode = 'nearest' )

# validation data 전처리 옵션 설정
valid_datagen = ImageDataGenerator(rescale = 1./255,
                                   rotation_range = 40,
                                   width_shift_range = 0.2,
                                   height_shift_range = 0.2,
                                   shear_range = 0.2,
                                   zoom_range = 0.2,
                                   horizontal_flip = True,
                                   vertical_flip = True,
                                   fill_mode = 'nearest' )

# test data 전처리 옵션 설정
test_datagen = ImageDataGenerator(rescale = 1./255 )
```

- rescale : 이미지의 픽셀값을 0과 1 사이로 정규화
- rotation_range : 이미지에 적용되는 회전범위^{각도} 지정
- width/height_shift_range : 이미지의 수평/수직 이동범위 지정
- shear_range : 이미지에 적용되는 임의의 변환범위^{각도} 지정
- zoom_range : 이미지에 적용되는 임의의 확대/축소 범위 지정
- horizontal/vertical_flip : 이미지를 수평/수직 뒤집기 여부 지정
- fill_mode : 이미지 변환의 결과로 생성된 새 픽셀을 채우는 데 사용할 방법 지정
nearest는 원본 이미지에서 가장 가까운 픽셀값으로 채

우기

② 데이터 전처리

```
# 'train_dir' 디렉토리에 저장된 이미지 전처리
train_generator = train_datagen.flow_from_directory(train_dir,
                                                    target_size = (150, 150),
                                                    batch_size = 20,
                                                    class_mode='binary')

# 'valid_dir' 디렉토리에 저장된 이미지 전처리
valid_generator = valid_datagen.flow_from_directory(valid_dir,
                                                    target_size = (150, 150),
                                                    batch_size = 20,
                                                    class_mode='binary')

# 'test_dir' 디렉토리에 저장된 이미지 전처리
test_generator = test_datagen.flow_from_directory(test_dir,
                                                  target_size = (150, 150),
                                                  batch_size = 20,
                                                  class_mode='binary',
                                                  shuffle = False)
```

Found 1014 images belonging to 2 classes.
Found 290 images belonging to 2 classes.
Found 144 images belonging to 2 classes.

② 데이터 전처리

- 데이터 확인

<pre>print(type(train_generator)) ## DirectoryIterator: ImageDataGenerator의 .flow_from_directory() 메서드가 반환하는 객체 ## 지정된 폴더에서 설정된 배치 크기만큼 이미지 데이터와 라벨을 한 묶음(batch)씩 생성</pre>	
<pre><class 'keras.src.legacy.preprocessing.image.DirectoryIterator'></pre>	

```
# DirectoryIterator는 Python의 이터레이터이므로, next() 함수를 사용해 데이터 확인 가능  
  
# DirectoryIterator는 (x, y) 형태의 튜플을 반환합니다.  
# x: 이미지 데이터 묶음 (Numpy 배열, (batch_size, height, width, channels) 형태)  
# y: 해당 이미지들의 라벨 묶음 (Numpy 배열)
```

② 데이터 전처리

- 데이터 확인

```
import matplotlib.pyplot as plt

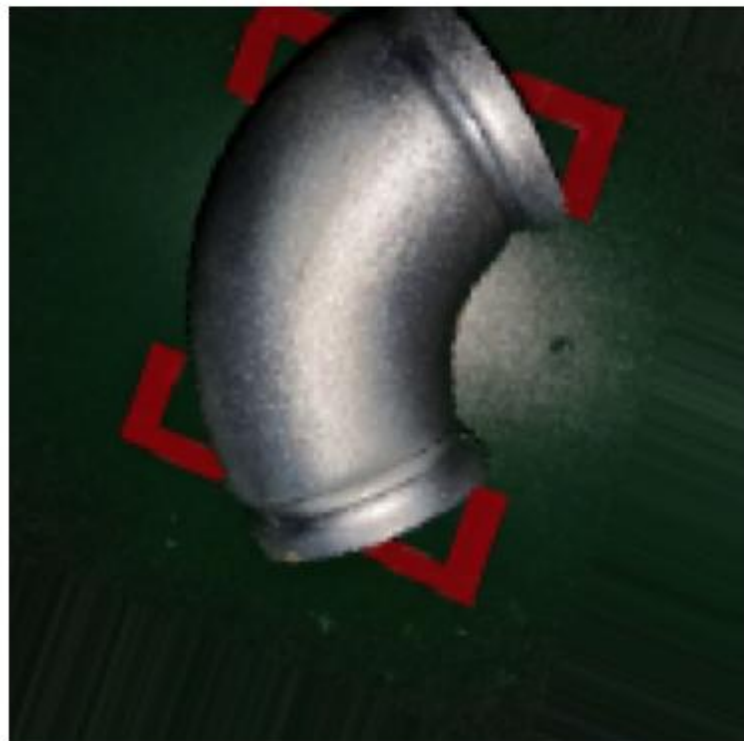
# next() 함수로 데이터 배치 한 개를 꺼냄
x_batch, y_batch = next(train_generator)

# 생성된 배치의 형태(shape) 확인
print('이미지 배치 shape:', x_batch.shape) # (20, 150, 150, 3)
print('라벨 배치 shape:', y_batch.shape)   # (20,)

# 배치에 포함된 첫 번째 이미지와 라벨 확인
print('첫 번째 이미지의 라벨:', y_batch[0])

# 첫 번째 이미지 시각화
plt.imshow(x_batch[0])
plt.axis('off')
plt.show()
```

이미지 배치 shape: (20, 150, 150, 3)
라벨 배치 shape: (20,)
첫 번째 이미지의 라벨: 1.0



② 데이터 전처리

- 유용한 속성들

```
# 총 이미지 개수
print('총 샘플 수:', train_generator.samples) # 또는 .n

# 배치 크기
print('배치 크기:', train_generator.batch_size)

# 클래스 이름과 인덱스 매핑 정보 (매우 유용)
# {'cat': 0, 'dog': 1}
print('클래스 인덱스:', train_generator.class_indices)

# 파일 경로 리스트
print('파일 경로:', train_generator.filepaths[0])
```

```
총 샘플 수: 1014
배치 크기: 20
클래스 인덱스: {'Plated_Image_Data_NG': 0, 'Plated_Image_Data_OK': 1}
파일 경로: /content/drive/MyDrive/_for_exercise/_PdM/chap06/Plated_Image_Data_Train/Plat
```

③ 모델링

- 합성곱 신경망 구성

```
from keras.models import Sequential
from keras.layers import Dense, Dropout, Flatten, Conv2D, MaxPooling2D, Input

model = Sequential()
model.add(Input(shape=(150, 150, 3)))

model.add(Conv2D(32, (3, 3), activation='relu', padding='same'))
model.add(MaxPooling2D((3, 3)))

model.add(Conv2D(128, (3, 3), activation='relu', padding='same'))
model.add(MaxPooling2D((2, 2)))

model.add(Conv2D(256, (3, 3), activation='relu', padding='same'))
model.add(MaxPooling2D((2, 2)))

model.add(Flatten())
model.add(Dropout(0.5))
model.add(Dense(128, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
```

③ 모델링

- 합성곱 신경망 구성

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 150, 150, 32)	896
max_pooling2d (MaxPooling2D)	(None, 50, 50, 32)	0
conv2d_1 (Conv2D)	(None, 50, 50, 128)	36,992
max_pooling2d_1 (MaxPooling2D)	(None, 25, 25, 128)	0
conv2d_2 (Conv2D)	(None, 25, 25, 256)	295,168
max_pooling2d_2 (MaxPooling2D)	(None, 12, 12, 256)	0
flatten (Flatten)	(None, 36864)	0
dropout (Dropout)	(None, 36864)	0
dense (Dense)	(None, 128)	4,718,720
dense_1 (Dense)	(None, 1)	129

Total params: 5,051,905 (19.27 MB)

Trainable params: 5,051,905 (19.27 MB)

Non-trainable params: 0 (0.00 B)

③ 모델링

- 컴파일 및 학습

```
model.compile(loss='binary_crossentropy',  
              optimizer='adam',  
              metrics=['accuracy'])  
  
history = model.fit(train_generator,  
                    epochs=10,  
                    steps_per_epoch=len(train_generator),  
                    validation_data=valid_generator,  
                    validation_steps=len(valid_generator))
```

④ 학습 결과 확인

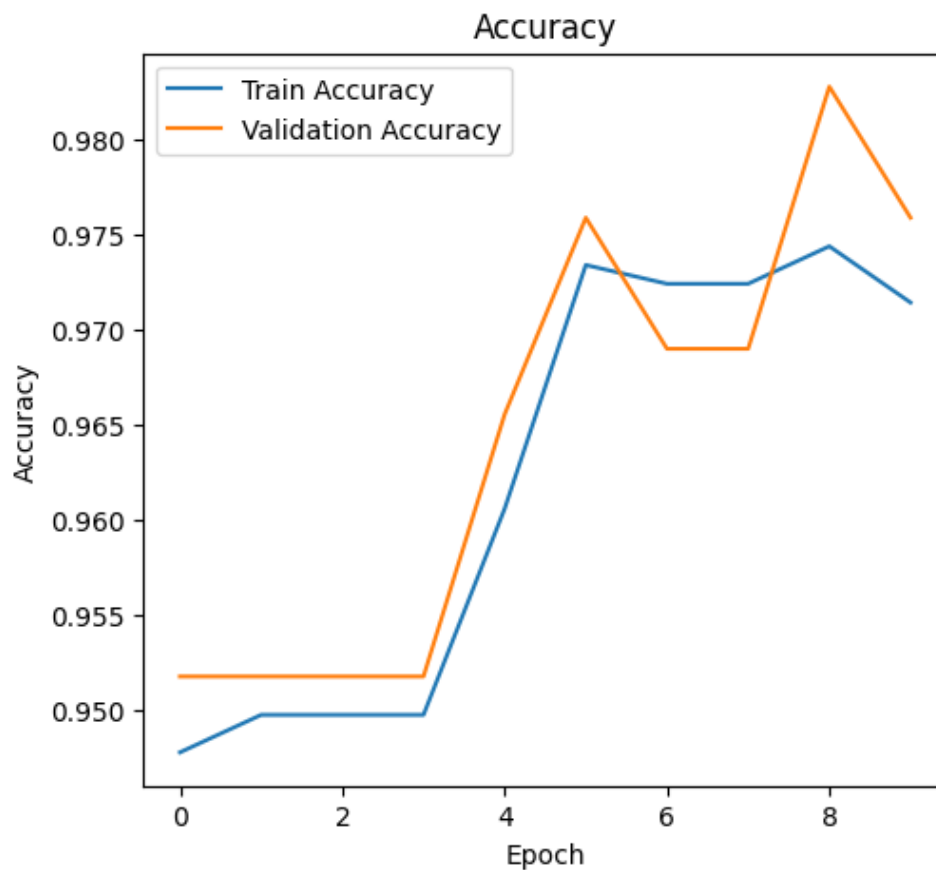
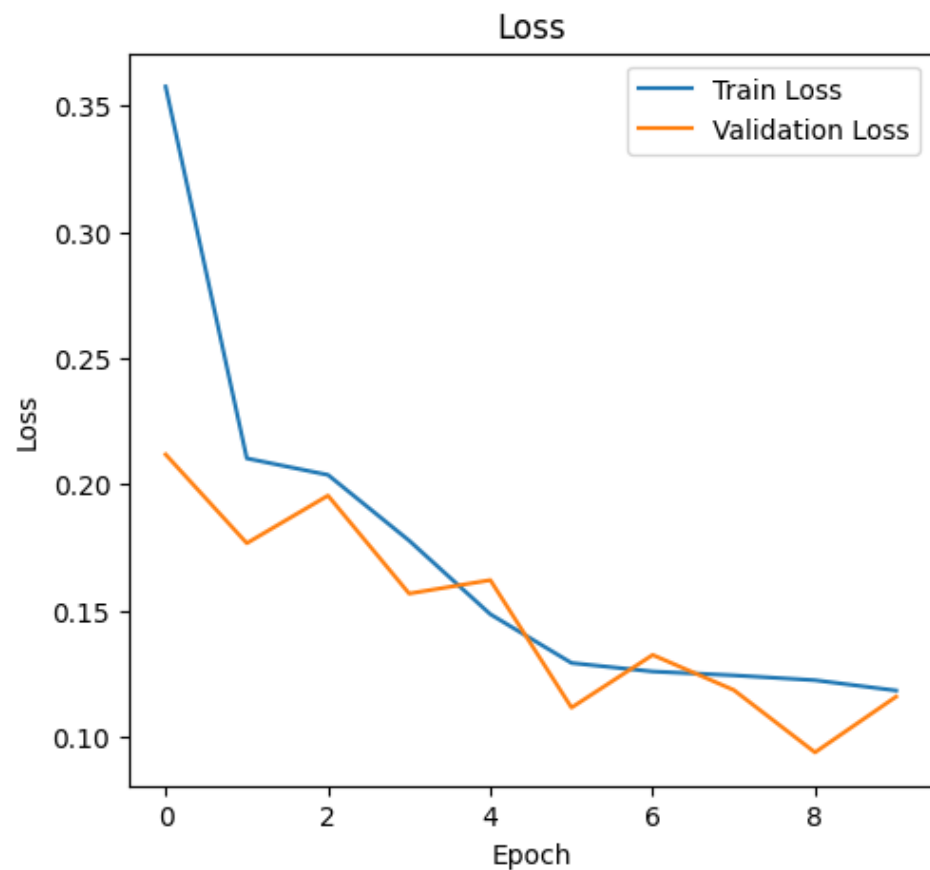
- 손실/정확도 그래프

```
# 학습 결과 시각화
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Loss')
plt.xlabel('Epoch'); plt.ylabel('Loss'); plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Accuracy')
plt.xlabel('Epoch'); plt.ylabel('Accuracy'); plt.legend()
plt.show()
```

④ 학습 결과 확인

- 손실/정확도 그래프



④ 학습 결과 확인

- 성능 지표 계산

```
import numpy as np
from sklearn.metrics import confusion_matrix, classification_report

## 테스트 데이터로 예측값 확인
y_preds = model.predict(test_generator, steps = test_generator.samples)
y_preds = np.rint(y_preds)
y_test = test_generator.classes

# 혼동행렬
cm_test = confusion_matrix(y_test, y_preds)
print('\nConfusion Matrix : test')
print(cm_test)

# 분류 성능 지표
print('\nReport : test')
print(classification_report(y_test, y_preds))
```

④ 학습 결과 확인

- 성능 지표 계산

Report : test				
	precision	recall	f1-score	support
0	1.00	0.71	0.83	7
1	0.99	1.00	0.99	137
accuracy			0.99	144
macro avg	0.99	0.86	0.91	144
weighted avg	0.99	0.99	0.99	144

⑤ 새로운 데이터에 대한 예측 구하기

```
new_dir = root_dir + '/Plated_Image_Data_New'

new_datagen = ImageDataGenerator(rescale = 1./255)
new_generator = new_datagen.flow_from_directory(new_dir,
                                                target_size = (150, 150),
                                                batch_size = 20,
                                                class_mode='binary',
                                                shuffle = False)
```

```
print("\n--- new data 예측하기 ---")
new_results = model.predict(new_generator)

print(new_results)
```

참고 자료

- 『제조AI빅데이터 분석 기법 (코딩편)』, 김일중 외 6인 공저, 보민출판사 (2024)
- 『혼자 공부하는 머신러닝 딥러닝』

끝!